

SZTUCZNA INTELIGENCJA

KLASYFIKACJA GESTÓW DŁONI W GRZE „KAMIEŃ, PAPIER, NOŻYCE” Z WYKORZYSTANIEM KONWOLUCYJNYCH SIECI NEURONOWYCH (CNN) w **PyTorch**

Wykonał: Filip Pelczarski

Nr Indeksu: 179582

Spis treści

1	Opis problemu	3
1.1	Cel projektu	3
1.2	Złożoność problemu i uzasadnienie podejścia	3
2	Część teoretyczna	4
2.1	Anatomia Sieci Neuronowych	4
2.1.1	Klasyczny model sztucznego neuronu	4
2.1.2	Ograniczenia wielowarstwowych sieci (MLP)	5
2.1.3	Rozwiązanie: Konwolucyjne Sieci Neuronowe (CNN)	5
2.2	Cyfrowa reprezentacja obrazu	5
2.3	Wielowymiarowa reprezentacja danych wejściowych	6
2.4	Architektura Konwolucyjnych Sieci Neuronowych (CNN)	7
2.4.1	Warstwy splotowe	8
2.4.2	Detekcja cech (Konwolucja)	8
2.4.3	Nieliniowość i funkcje aktywacji	10
2.4.4	Redukcja wymiarowości przestrzennej (Max Pooling)	11
2.5	Klasyfikator i warstwy w pełni połączone (Dense)	11
2.5.1	Transformacja wektorowa (Flattening)	12
2.5.2	Zapobieganie przeuczeniu (Dropout)	12
2.5.3	Warstwa wyjściowa i funkcja Softmax	12
2.6	Proces uczenia i optymalizacja	13
2.6.1	Propagacja w przód (Forward Propagation)	13
2.6.2	Kategoryczna Entropia Krzyżowa jako funkcja straty	13
2.6.3	Algorytm propagacji wstecznej (Backpropagation)	14
2.6.4	Optymalizator AdamW	15
2.6.5	Mechanika pętli uczącej i zarządzanie danymi	16
2.7	Metryki ewaluacji i Macierz Pomyłek	16
3	Przygotowanie i analiza danych	18
3.1	Charakterystyka i struktura zbioru danych	18
3.2	Podział zbioru	19
3.3	Wstępne przetwarzanie i normalizacja	19
3.4	Proces augmentacji obrazów	20
3.5	Mechanizmy ładowania zbiorów	22
4	Architektura i implementacja modelu CNN	24
4.1	Struktura opracowanej sieci CNN	24
4.2	Parametryzacja architektury w konstruktorze (<code>__init__</code>)	24
4.3	Przepływ danych i hierarchia cech (Forward Pass)	26
4.4	Pętla ucząca i zarządzanie optymalizacją (Training Loop)	27
4.5	Adaptacyjne harmonogramowanie kroku uczenia (Cosine Annealing)	30
4.6	Mechanizm Checkpointingu i Ewaluacja Ciągła	30
5	Badania eksperymentalne i optymalizacja hiperparametrów	31
5.1	Metodologia badań i środowisko testowe	31
5.2	Etap I: Architektura przestrzenna i ekstrakcja cech	32
5.3	Etap II: Dynamika uczenia i stabilność gradientowa	33

5.4	Etap III: Regularyzacja i ochrona przed przeuczeniem	34
5.5	Weryfikacja końcowa i konfiguracja modelu optymalnego	34
6	Weryfikacja modelu w warunkach rzeczywistych	38
6.1	Weryfikacja na danych znanych i autorskich	38
6.2	Analiza zjawiska przesunięcia domeny (<i>Domain Shift</i>)	39
7	Podsumowanie i wnioski	40
8	Bibliografia	41

1 Opis problemu

1.1 Cel projektu

Celem pracy jest projekt, implementacja oraz optymalizacja sieci konwolucyjnej (CNN) w procesie klasyfikacji gestów dłoni w grze „Kamień, Papier, Nożyce”. Opracowany model stanowi inżynierski dowód skuteczności głębokich architektur w zadaniach ekstrakcji cech wizualnych, dostarczając gotowy moduł decyzyjny możliwy do implementacji w systemach sterowania bezdotykowego.

1.2 Złożoność problemu i uzasadnienie podejścia

Klasyfikacja obrazów gestów w czasie rzeczywistym napotyka na szereg wyzwań związanych z dużą wariancją danych wejściowych. Niestabilne warunki oświetleniowe, różnorodne tło oraz zmienna geometria dłoni sprawiają, że podejścia oparte na przetwarzaniu koloru skóry czy detekcji krawędzi są wysoce podatne na błędy i mają ograniczoną skalowalność.

W obliczu tych ograniczeń, jedyną skuteczną metodą jest zastosowanie głębokich sieci neuronowych. W przeciwieństwie do metod klasycznych, modele typu CNN (Convolutional Neural Networks) realizują proces uczenia się hierarchicznych reprezentacji cech bezpośrednio z surowych danych wejściowych. Podejście to zapewnia wysoką odporność na szum środowiskowy oraz niezmienność translacyjną, co jest kluczowe dla poprawności rozpoznawania gestów niezależnie od pozycji dłoni w kadrze.

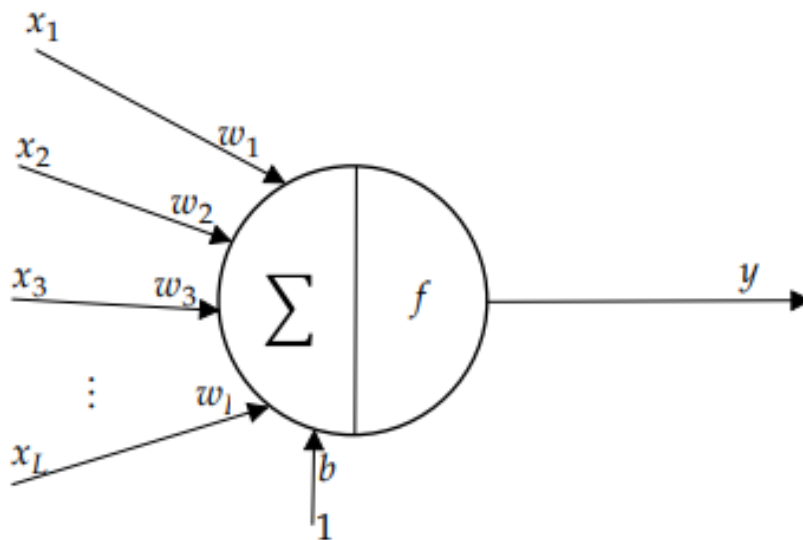
2 Część teoretyczna

2.1 Anatomia Sieci Neuronowych

Historycznym i matematycznym fundamentem współczesnych architektur wizyjnych są modele pojedynczych komórek nerwowych. Zanim technologia obliczeniowa pozwoliła na efektywne przetwarzanie wielowymiarowych tensorów przestrzennych, podstawy sztucznej inteligencji oparto na uproszczonym modelu sztucznego neuronu, wywodzącym się z wczesnych prac badawczych McCullocha i Pittsa.

2.1.1 Klasyczny model sztucznego neuronu

Podstawową jednostką obliczeniową każdej sztucznej sieci nerwowej jest model neuronu. Jego zadaniem jest odbiór sygnałów wejściowych, ich zważenie, zsumowanie oraz wygenerowanie odpowiedzi wyjściowej.



Rysunek 1: Model neuronu McCullocha-Pittsa.

Sygnał wyjściowy y pojedynczego neuronu zdefiniowany jest równaniem:

$$y = f \left(\sum_{j=1}^n w_j \cdot x_j + b \right) \quad (1)$$

Gdzie poszczególne elementy oznaczają:

- x_j – j -ty sygnał wejściowy (reprezentujący np. cechę z danych).
- w_j – współczynnik wagowy (waga). Określa on, jak duży wpływ ma konkretny sygnał na ostateczną decyzję neuronu. Są to wartości swobodne, modyfikowane podczas uczenia.
- b – obciążenie (bias), czyli parametr przesuwający próg aktywacji neuronu, niezależny od wejść.
- f – funkcja aktywacji, wprowadzająca nieliniowość.

2.1.2 Ograniczenia wielowarstwowych sieci (MLP)

Pojedynczy neuron rozwiązuje jedynie problemy separowalne liniowo. Aby modelować skomplikowane zależności, neurony łączą się w struktury określane jako Wielowarstwowe Sieci Perceptronowe (MLP). Choć stanowią one potężne narzędzie w analizie danych ustrukturyzowanych (np. tabelarycznych), ich zastosowanie w zadaniach widzenia komputerowego obarczone jest dwiema fundamentalnymi wadami:

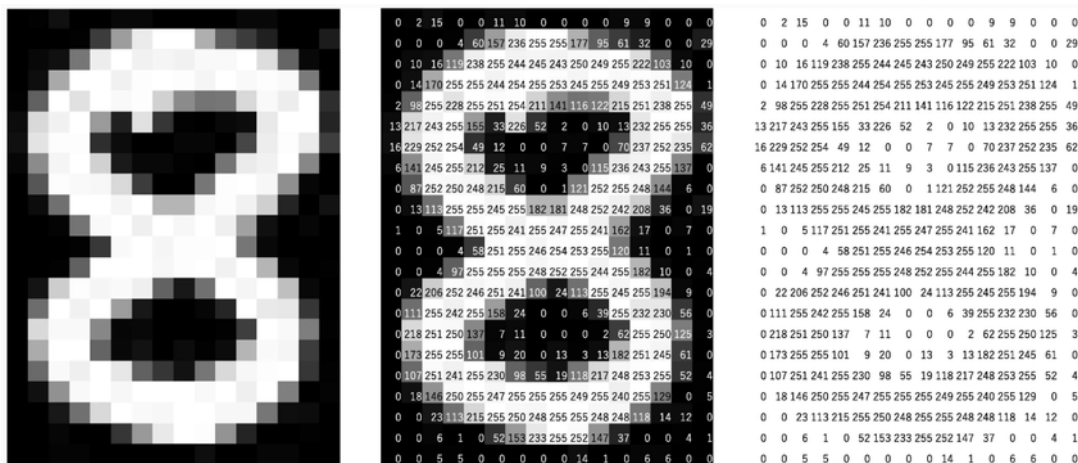
1. **Eksplzja parametrów:** W klasycznej sieci MLP każdy neuron warstwy ukrytej jest połączony z każdym pojedynczym pikselem obrazu wejściowego. Nawet dla zdjęć o stosunkowo niskiej rozdzielczości, połączenie macierzy obrazu zaledwie z jedną małą warstwą ukrytą generuje miliony wag. Wymaga to absurdalnej ilości pamięci i natychmiastowo prowadzi do przeuczenia modelu (Overfitting).
2. **Destrukcja topologii przestrzennej:** Klasyczny perceptron operuje wyłącznie na płaskich wektorach danych (1D). Zmusza to algorytm do mechanicznego spłaszczenia dwuwymiarowego obrazu przed rozpoczęciem analizy. Proces ten bezpowrotnie niszczy informacje o relacjach przestrzennych, sąsiedztwie pikseli i naturalnej geometrii obiektów.

2.1.3 Rozwiązanie: Konwolucyjne Sieci Neuronowe (CNN)

Odpowiedzią na powyższe bariery technologiczne są architektury konwolucyjne. Zamiast analizować wszystkie piksele naraz za pomocą płaskich wektorów, wykorzystują one matematyczną operację splotu do przestrzennego skanowania obrazu zachowanego w jego natywnej, wielowymiarowej formie (tensorowej). Dodatkowo, dzięki mechanizmowi współdzielenia wag (te same filtry są aplikowane na całym obrazie), architektura ta drastycznie redukuje liczbę parametrów, co umożliwia budowanie niezwykle głębokich i wydajnych modeli.

2.2 Cyfrowa reprezentacja obrazu

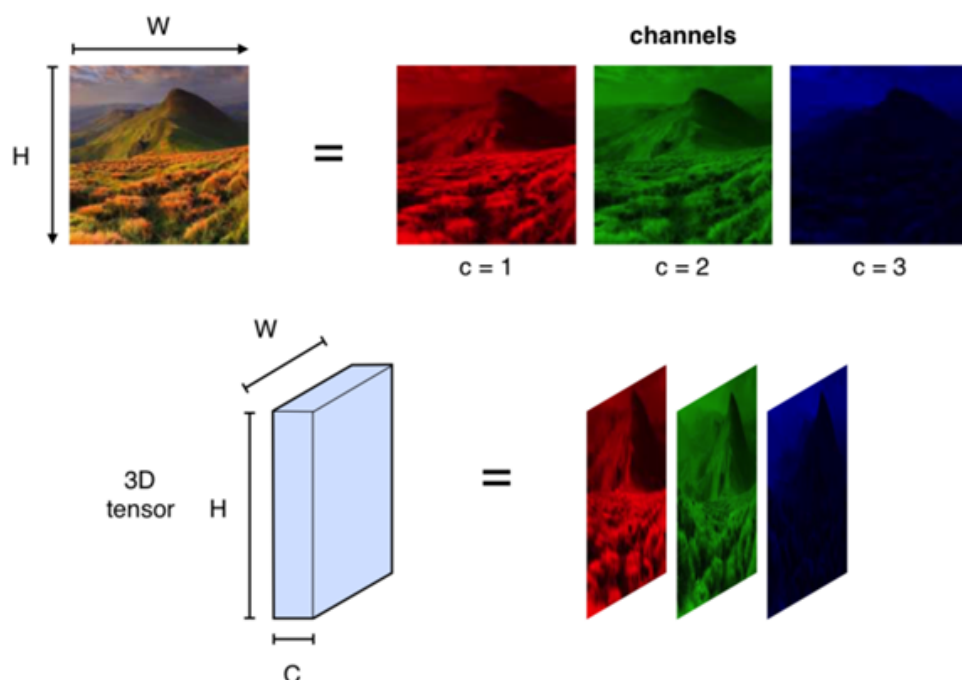
Z perspektywy człowieka rozpoznawanie obiektów na zdjęciu jest natychmiastowe i podświadome. Patrząc na dłoń pokazującą gest „nożyc”, ludzki mózg bezbłędnie ignoruje tło, analizuje ułożenie palców i kategoryzuje kształt. Z kolei dla komputera to samo zdjęcie to po prostu dwuwymiarowa macierz liczb, które reprezentują jasność poszczególnych pikseli.



Rysunek 2: Czarno-Biały, fizyczny obraz jako ustrukturyzowana macierz wartości liczbowych.

Zjawisko to doskonale obrazuje proces cyfrowego próbkowania. Im głębiej algorytm analizuje zdjęcie, tym bardziej rzeczywisty kształt zanika, ustępując miejsca gigantycznej tabeli (siatce pikseli). Z fizycznego punktu widzenia każda komórka takiej siatki przechowuje wartość liczbową, która najczęściej mieści się w przedziałach od 0 (absolutna czern) do 255 (pełna jasność).

W przypadku powszechnie stosowanych obrazów kolorowych sytuacja staje się jeszcze bardziej złożona. Standardowe cyfrowe zdjęcie nie jest jedną, płaską tabelą, lecz wynikiem nałożenia na siebie trzech niezależnych warstw, odpowiadających za barwy podstawowe: czerwoną, zieloną i niebieską (przestrzeń barw RGB). Na przykład wypadkowy kolor żółty dla maszyny nie istnieje jako odrębna wartość – jest wynikiem wysokiego nasycenia wartości w macierzy czerwonej i zielonej, przy jednoczesnym zerowym natężeniu na warstwie niebieskiej.



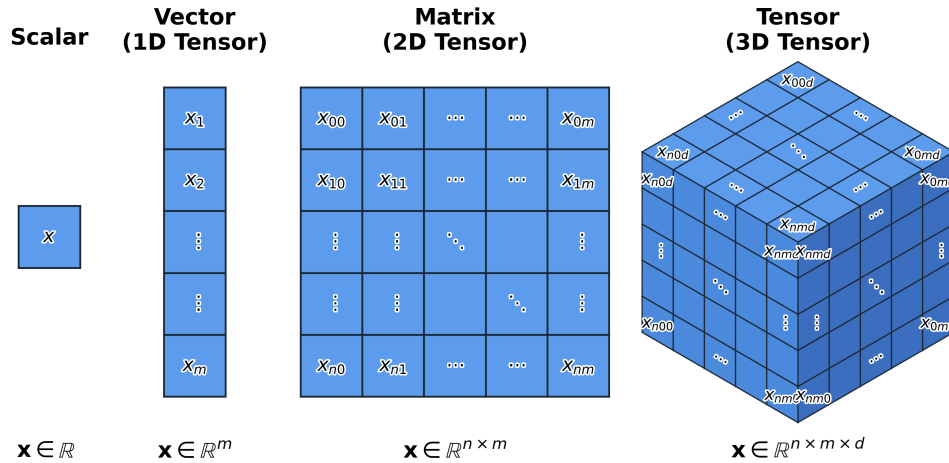
Rysunek 3: Dekompozycja kolorowego obrazu na niezależne kanały barw (RGB) oraz ich integracja w trójwymiarowy tensor danych.

2.3 Wielowymiarowa reprezentacja danych wejściowych

Z perspektywy systemu wizyjnego, zdjęcie nie jest obiektem fizycznym, lecz wielowymiarową strukturą danych liczbowych. Aby sieć neuronowa mogła sprawnie przeprowadzać równoległe obliczenia na tak dużych zbiorach informacji, organizuje się je w ustandaryzowane kontenery. W inżynierii uczenia maszynowego nazewnictwo tych struktur wynika bezpośrednio z liczby posiadanych przez nie wymiarów (osi przestrzennych):

Najmniejszą, niepodzielną jednostką informacji – na przykład wartością jasności pojedynczego, czerwonego piksela – jest **skalar**, będący pojedynczą liczbą (wymiar 0D). Gdybyśmy wyodrębnili z obrazu tylko jeden poziomy rząd takich pikseli, otrzymalibyśmy jednowymiarowy ciąg danych nazywany **wektorem** (1D).

Idąc o krok dalej, pojedynczy pełny kanał koloru (np. cała czerwona warstwa zdjęcia, posiadająca swoją szerokość i wysokość) tworzy płaską, dwuwymiarową tablicę, czyli **macierz** (2D). Ostatecznie, gdy nałożymy na siebie wszystkie trzy macierze koloru (RGB), uzyskujemy złożoną strukturę trójwymiarową określaną mianem **tensora** (3D).



Rysunek 4: Ewolucja wymiarowości struktur danych: od pojedynczego skalar (0D) do trójwymiarowego tensora (3D).

W kontekście implementowanego w projekcie modelu CNN, każda wejściowa klatka ze zdjęciem gestu o ujednoliconej rozdzielczości 128×128 pikseli jest fizycznie reprezentowana w pamięci karty graficznej jako trójwymiarowy tensor o wymiarach $3 \times 128 \times 128$. Oznacza to, że z perspektywy wejścia sieci, pojedynczy gest dłoni nie jest płaskim obrazkiem, lecz zbitą bryłą danych składającą się dokładnie z 49 152 niezależnych wartości liczbowych ($3 \cdot 128 \cdot 128$).

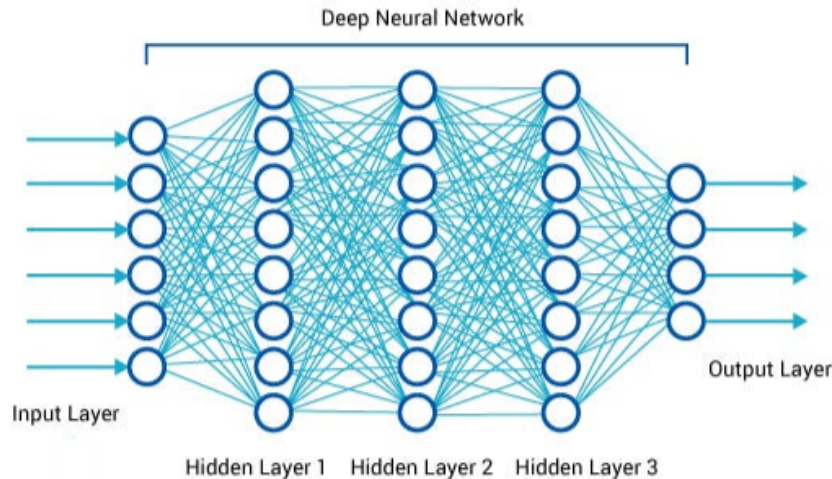
Zachowanie obrazu wejściowego w jego natywnej, wielowymiarowej formie tensorowej jest warunkiem koniecznym dla skutecznego uczenia. Próba wektoryzacji (spłaszczenia) tej struktury na samym początku przetwarzania zniszczyłaby kluczowe informacje o relacjach przestrzennych, sąsiedztwie pikseli i geometrii dłoni. To właśnie wymóg operowania na nienaruszonych, trójwymiarowych tensorach stanowi fundamentalny powód, dla którego zrezygnowano z klasycznych sieci gęstych na rzecz architektury konwolucyjnej, przystosowanej do przestrzennej analizy danych.

2.4 Architektura Konwolucyjnych Sieci Neuronowych (CNN)

Konwolucyjne Sieci Neuronowe (CNN) stanowią paradygmat w dziedzinie głębokiego uczenia, w którym klasyczna, gęsta struktura połączeń zostaje zastąpiona operacją splotu. Kluczowym założeniem tej architektury jest przyjęcie zasady *lokalności cech* – założenia, iż każdy istotny wzorzec wizualny (krawędź, kształt czy tekstura) jest zdefiniowany przez relacje pomiędzy sąsiadującymi pikselami, a nie przez globalną analizę całego obrazu. W przeciwieństwie do architektur w pełni połączonych, CNN wymuszają na sieci automatyczne wykrywanie tych zależności poprzez przesuwany mechanizm filtrów, co drastycznie redukuje liczbę wymaganych wag i pozwala na wydajne przetwarzanie informacji wizualnych.

2.4.1 Warstwy splotowe

Podstawowym blokiem budulcowym architektury CNN jest warstwa splotowa. Fizycznie nie składa się ona z płaskich rzędów neuronów, lecz z trójwymiarowych bloków wagowych, nazywanych **filtrami** (lub jądrami splotu). Każdy taki filtr (najczęściej o rozmiarze przestrzennym 3×3 piksele) to niewielka, ucząca się macierz, która specjalizuje się w wyszukiwaniu konkretnej cechy wizualnej.



Rysunek 5: Schemat architektury sieci głębokiej oraz przepływ danych pomiędzy warstwami.

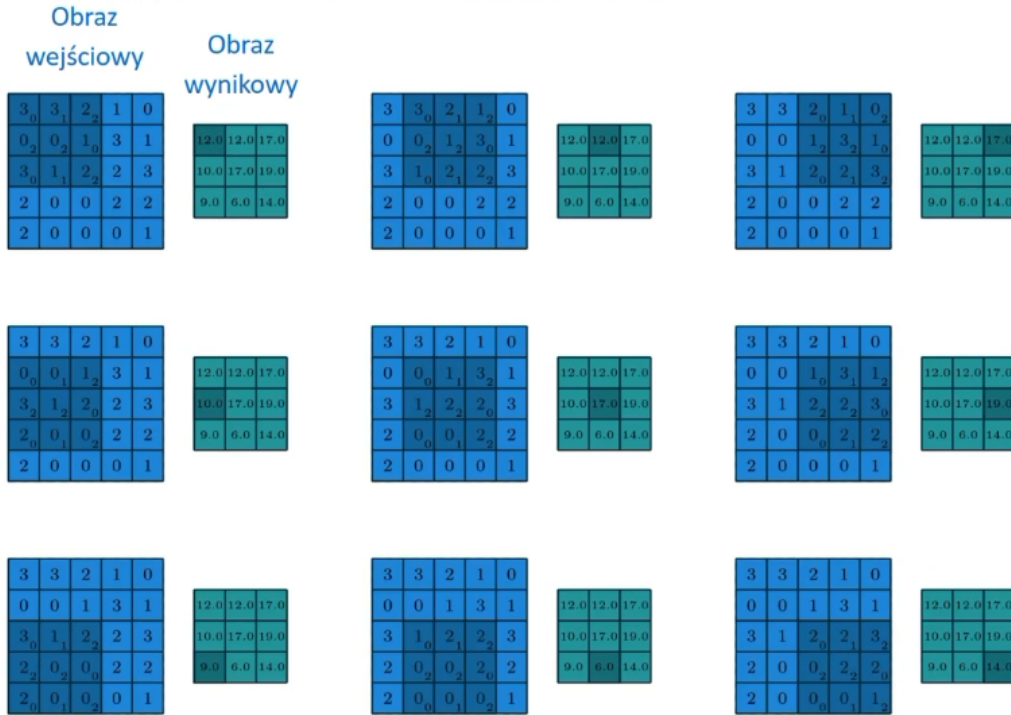
Architektura sieci w omawianym projekcie celowo podwaja liczbę filtrów w kolejnych warstwach ($16 \rightarrow 32 \rightarrow 64$). Wynika to z konieczności budowania **hierarchii cech**. Płytkie warstwy (posiadające 16 filtrów) skanują surowy obraz w poszukiwaniu trywialnych, niskopoziomowych struktur – pionowych krawędzi czy gradientów cieni. Głębsze warstwy (64 filtry) nie patrzą już na surowe piksele, lecz łączą te wykryte wcześniej proste krawędzie w złożone, wielowymiarowe obiekty – na przykład zarys zgiętego kciuka charakterystycznego dla gestu „kamień”.

2.4.2 Detekcja cech (Konwolucja)

Właściwy proces detekcji realizowany jest poprzez matematyczną operację splotu (konwolucji). Polega ona na iteracyjnym przesuwaniu filtra po całej powierzchni wejściowego tensora i obliczaniu sumy iloczynów pokrywających się wartości. Dyskretna operacja splotu dwuwymiarowego, generująca pojedynczy element $y[i, j]$ na nowej mapie cech, wyraża się wzorem:

$$y[i, j] = \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} x[i+u, j+v] \cdot w[u, v] \quad (2)$$

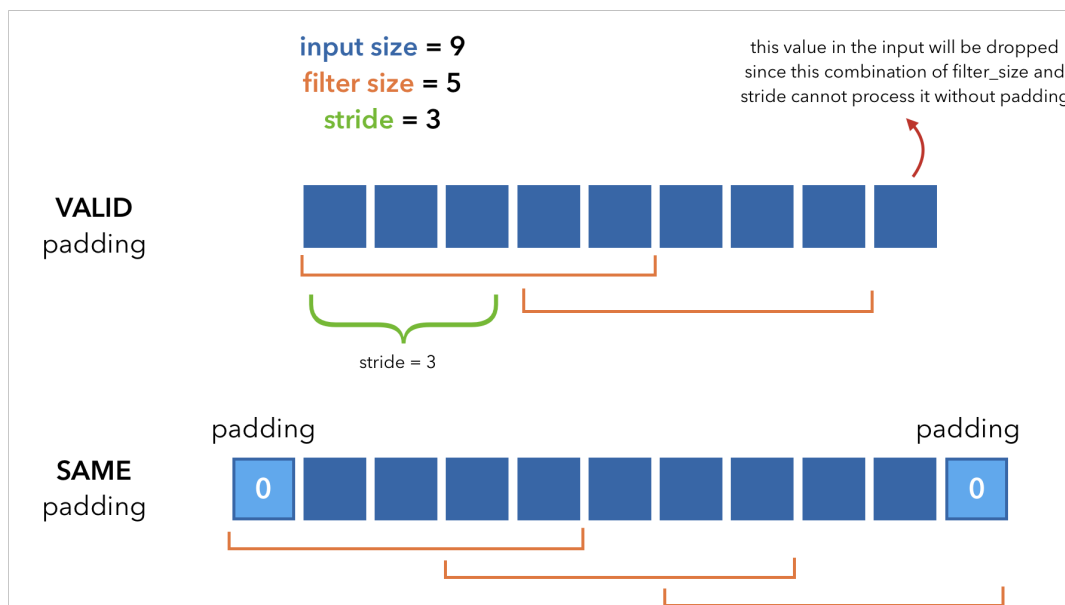
Gdzie x to nakładający się fragment obrazu wejściowego, w to wagi przesuwanego filtra, a M i N to jego wymiary. W praktyce, jeśli fragment obrazu silnie koreluje z wagami filtra (np. obie struktury układają się w pionową linię), wynik mnożenia jest wysoki – w ten sposób sieć „zauważa” szukany wzorzec.



Rysunek 6: Mechanika przesuwania filtra (kernel) po macierzy wejściowej i generowanie mapy cech.

Parametryzacja procesu skanowania opiera się na dwóch głównych hiperparametrach, które ściśle determinują zachowanie filtra oraz ostateczny wymiar przestrzenny generowanej mapy cech:

- **Padding (P):** Sztuczny margines z zer (ramka) dodawany na krawędziach obrazu. Służy on zapobieganiu kurczeniu się mapy lub wręcz utracie skrajnych danych. Jak ukazuje uproszczony, jednowymiarowy przykład na Rysunku 7, brak paddingu (*Valid Padding*) w połączeniu z określonym rozmiarem filtra może sprawić, że skrajne piksele zostaną całkowicie zignorowane przez sieć. Zastosowanie outline'a z zer (*Same Padding*) pozwala zachować spójność strukturalną i przetworzyć 100% wejściowych danych.
- **Stride (S):** Parametr kroku określający, o ile pikseli przesuwany jest filtr po wykonaniu obliczenia. Domyślny krok $S = 1$ zapewnia gęste próbkowanie, natomiast wyższe wartości (np. $S = 3$ widoczne na Rysunku 7) drastycznie redukują rozmiar danych wyjściowych, wymuszając skróconą analizę.



Rysunek 7: Uproszczony model jednowymiarowy obrazujący wpływ parametrów Stride oraz Padding na utratę danych skrajnych. Porównanie strategii VALID (ucięcie) i SAME (zera wokół obrazu).

Znając te parametry oraz wejściową szerokość obrazu (W) i długość boku filtra (K), ostateczny wymiar wyjściowej mapy cech (O) wylicza się ze wzoru:

$$O = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1 \quad (3)$$

2.4.3 Nieliniowość i funkcje aktywacji

Gdyby architektura składała się wyłącznie z warstw splotowych, sieć byłaby matematycznie równoważna jednej, gigantycznej funkcji liniowej, całkowicie niezdolnej do modelowania skomplikowanych kształtów i krzywizn dłoni. Aby rozwiązać ten problem, na wyjściu każdego splotu aplikowana jest funkcja aktywacji.

W niniejszym projekcie wykorzystano nowoczesną funkcję **SiLU (Sigmoid Linear Unit)**. Działa ona jak zaawansowana bramka sygnałowa: wzmacnia i lekko zaokrągla silne wartości (wykryte cechy), a gładko wygasza sygnały negatywne. Pozwala to sieci na wysoce nieliniową interpretację zgromadzonych map cech, zapobiegając jednocześnie problemowi "martwych neuronów" (charakterystycznemu dla klasycznej funkcji ReLU).

2.4.4 Redukcja wymiarowości przestrzennej (Max Pooling)

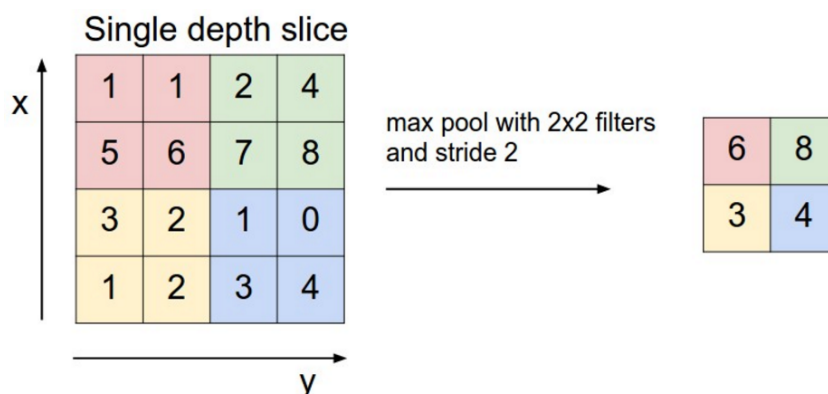
Kolejnym kluczowym krokiem po wykonaniu splotu i aktywacji jest redukcja ilości danych, czyli tzw. *Downsampling*. Operacja ta jest realizowana przez warstwę **Max Pooling**.

Jej działanie polega na podziale mapy cech na niewielkie, nienakładające się na siebie obszary (zazwyczaj o rozmiarze 2×2 piksele) i wybraniu z każdego bloku wyłącznie jednej, największej wartości. Reszta danych jest bezpowrotnie odrzucana.

$$y[i, j, c] = \max_{0 \leq u, v < F} x(S \cdot i + u, S \cdot j + v, c) \quad (4)$$

Gdzie:

- x – wejściowa mapa cech,
- S – krok przesunięcia okna (*Stride*),
- F – rozmiar okna filtra redukującego,
- c – indeks analizowanego kanału (mapy cech), na którym operacja wykonywana jest niezależnie.



Rysunek 8: Zasada działania warstwy Max Pooling redukującej rozdzielczość przestrzenną mapy cech z wykorzystaniem bloku 2×2 i kroku 2.

Taka agresywna kompresja pełni dwie krytyczne funkcje w projekcie. Po pierwsze, zapobiega wykładniczemu przepełnieniu pamięci karty graficznej, pozwalając na stosowanie większej liczby filtrów w głębszych warstwach. Po drugie, zapewnia tzw. **niezmienniczość na translację** – jeśli dłoń na zdjęciu przesunie się o kilka pikseli w bok, z perspektywy bloku 2×2 w warstwie Max Pooling najwyższa aktywacja pozostanie w tym samym rejonie, uodparniając sieć na drobne przesunięcia kadru.

2.5 Klasyfikator i warstwy w pełni połączone (Dense)

Część konwolucyjna sieci (warstwy splotowe i redukcyjne) pełni wyłącznie rolę "oczu - ekstraktora przestrzennego rozpoznającego struktury wizualne. Aby na ich podstawie podjąć ostateczną decyzję (wydać wyrok: kamień, papier czy nożyce), dane te muszą trafić do wielowarstwowego perceptronu (MLP), pełniącego funkcję "mózgu decyzyjnego", czyli **Klasyfikatora**.

2.5.1 Transformacja wektorowa (Flattening)

Głównym problemem integracji części splotowej z klasyfikatorem jest niezgodność wymiarów. Klasyfikator zbudowany z warstw w pełni połączonych (Dense) dokonuje obliczeń na podstawie fundamentalnego równania algebry liniowej ($y = Wx + b$), gdzie wejście x **musi** być jednowymiarowym wektorem.

Z tego powodu, trójwymiarowy tensor map cech zostaje poddany operacji spłaszczania (**Flattening**). Polega to na mechanicznym rozpleceniu bloków 3D do pojedynczego ciągu liczbowego (1D). Relację transformacji między oryginalnym położeniem cechy w tensorze a jej nowym indeksem k w wektorze opisuje wzór:

$$k = i \cdot (W \cdot C) + j \cdot C + c \quad (5)$$

Gdzie i, j to indeksy przestrzenne cechy, c to numer kanału, a W i C to odpowiednio całkowita szerokość oraz liczba kanałów mapy. Tak przygotowany wektor trafia do sieci gęstej, gdzie neurony uczą się korelować określone w nim aktywacje z ostateczną klasą gestu.

2.5.2 Zapobieganie przeuczeniu (Dropout)

Potężna sieć gęsta posiada miliony wag, co stwarza ogromne ryzyko zjawiska tzw. koadaptacji. Występuje ono, gdy tylko kilka dominujących neuronów w klasyfikatorze podejmuje całą pracę, ucząc się na pamięć specyficznych cech zbioru treningowego (przeuczenie), podczas gdy reszta neuronów ignoruje sygnały.

Aby wymusić na sieci naukę uogólnioną i odporność, w klasyfikatorze zaimplementowano warstwę **Dropout**. Działa ona jak drastyczny mechanizm regularyzacyjny – podczas każdej iteracji treningowej losowo "odcina prąd" (zeruje wagi i sygnały) zadeklarowanej części neuronów (w projekcie ustalono ten próg na 20%). W efekcie, pozostałe przy życiu 80% neuronów musi przejąć odpowiedzialność za rozpoznanie gestu. Wymusza to na klasyfikatorze budowanie rozproszonych i wysoce niezależnych ścieżek decyzyjnych.

2.5.3 Warstwa wyjściowa i funkcja Softmax

Na samym końcu architektury znajduje się warstwa wyjściowa, posiadająca dokładnie tyle węzłów, ile wynosi liczba klas w problemie (czyli 3 węzły). Surowe wartości liczbowe generowane przez te neurony (tzw. logity z) są liczbami rzeczywistymi, które same w sobie nie niosą czytelnej informacji o pewności sieci.

W celu ich interpretacji stosuje się funkcję **Softmax**, przekształcającą logity w znormalizowany rozkład prawdopodobieństwa:

$$P(y = j | \mathbf{z}) = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}} \quad (6)$$

Wykorzystanie potęgi liczby Eulera (e^z) sprawia, że faworyzowane są najwyższe surowe aktywacje, a dzięki podzieleniu przez sumę, wyniki wszystkich trzech neuronów skalowane są do przedziału $(0, 1)$ tak, by łącznie dawały równe 1 (100%). Pozwala to algorytmom optymalizacyjnym (opisanym w sekcji 2.5) na bezbłędne policzenie błędu i aktualizację wag.

2.6 Proces uczenia i optymalizacja

Zaprojektowanie architektury sieci i przygotowanie danych stanowi jedynie przygotowanie środowiska. Aby model zyskał zdolność rozpoznawania gestów dłoni, konieczne jest przeprowadzenie procesu optymalizacji jego wewnętrznych wag. Proces ten ma charakter iteracyjny i opiera się na ciągłym minimalizowaniu błędów predykcji.

2.6.1 Propagacja w przód (Forward Propagation)

Pierwszym etapem każdej iteracji jest przepuszczenie danych wejściowych przez całą architekturę sieci – od warstw spłotowych aż do warstwy wyjściowej. Na tym etapie model działa w trybie wnioskowania. Wyodrębnia cechy ze zdjęcia (konwolucja), kompresuje je (pooling), spłaszcza do wektora, a klasyfikator wylicza surowe prawdopodobieństwa. Etap ten kończy się wygenerowaniem wektora predykcji dla badanej próbki – na przykład wynikiem wskazującym 85% pewności na klasę „Kamień”.

2.6.2 Kategoryczna Entropia Krzyżowa jako funkcja straty

Sieć musi zmierzyć, w jakim stopniu jej predykcja mija się z prawdziwą etykietą obrazu. Z racji rozwiązywania problemu klasyfikacji wieloklasowej, uniwersalnym standardem jest zastosowanie funkcji Kategorycznej Entropii Krzyżowej (Categorical Cross-Entropy Loss). Wyraża się ona równaniem:

$$L = - \sum_{c=1}^K y_c \log(\hat{y}_c) \quad (7)$$

Gdzie:

- K – całkowita liczba klas (w projekcie $K = 3$).
- y_c – wartość rzeczywista, zakodowana w formacie *One-Hot Encoding* (1 dla prawdziwej klasy, 0 dla pozostałych).
- $\hat{y}_c$ – prawdopodobieństwo przynależności do danej klasy przewidziane przez sieć.

Dzięki użyciu logarytmu, funkcja ta w sposób wykładniczy karze model za dużą pewność przy jednoczesnej błędnej odpowiedzi. Obliczona wartość straty (L) stanowi punkt wyjścia dla algorytmów optymalizacyjnych.

2.6.3 Algorytm propagacji wstecznej (Backpropagation)

Obliczenie całkowitego błędu sieci (np. poprzez funkcję straty E) to zaledwie diagnoza problemu. Zasadniczym wyzwaniem uczenia maszynowego jest przypisanie części tego błędu każdej pojedynczej wadze w_{ij} w wielomilionowej architekturze modelu. Dokonuje się tego za pomocą algorytmu propagacji wstecznej, który w ujęciu inżynierskim stanowi praktyczną implementację reguły łańcuchowej (ang. *Chain Rule*).

Zgodnie z ugruntowaną notacją klasycznych modeli neuronowych (z których wywodzą się współczesne architektury konwolucyjne), wyjście i -tego neuronu (oznaczane jako y_i) zależy od sumy ważonej jego wejść z_i , która z kolei uwikłana jest w wagi w_{ij} i sygnały z warstwy poprzedniej x_j :

$$z_i = \sum_{j=1}^L w_{ij} \cdot x_j + b_i \quad (8)$$

$$y_i = f(z_i) \quad (9)$$

Gdzie b_i to parametr obciążenia (ang. *bias*), który pozwala na niezależne przesunięcie progu aktywacji danego neuronu, zwiększając elastyczność sieci.

Aby zmodyfikować wagę metodą spadku wzdłuż gradientu (minimalizując sumaryczny błąd E), inżynier musi wyznaczyć wektor przyrostu Δw_{ij} , który proporcjonalny jest do pochodnej cząstkowej $\frac{\partial E}{\partial w_{ij}}$. Uwzględniając wielokrotne zagnieżdżenie funkcji, pochodną tę rozkłada się za pomocą reguły łańcuchowej na trzy łatwe do obliczenia człony:

$$\frac{\partial E}{\partial w_{ij}} = \underbrace{\frac{\partial E}{\partial y_i}}_{\text{Błąd na wyjściu}} \cdot \underbrace{\frac{\partial y_i}{\partial z_i}}_{\text{Pochodna aktywacji}} \cdot \underbrace{\frac{\partial z_i}{\partial w_{ij}}}_{\text{Wejście z warstwy poprzedniej}} \quad (10)$$

Zgodnie z analizą matematyczną przepływu sygnału:

1. Człon pierwszy reprezentuje różnicę między sygnałem oczekiwanym a otrzymanym ($\frac{\partial E}{\partial y_i} \propto y_i - \hat{y}_i$).
2. Człon drugi to uwarunkowana nieliniowo podatność neuronu na zmiany (np. dla klasycznej funkcji tangensa hiperbolicznego wynosi on $1 - y_i^2$).
3. Człon trzeci redukuje się algebraicznie do samej wartości sygnału wejściowego $\frac{\partial z_i}{\partial w_{ij}} = x_j$.

Aby uniknąć wielokrotnego rozwiązywania tych samych równań i przyspieszyć operacje macierzowe, dwa pierwsze człony (pochodną błędu oraz pochodną aktywacji) grupuje się w zunifikowany **lokalny sygnał błędu** δ_i :

$$\delta_i = \frac{\partial E}{\partial y_i} \cdot \frac{\partial y_i}{\partial z_i} \quad (11)$$

Dzięki takiemu zabiegowi, ostateczna reguła aktualizacji wagi – tożsama z uogólnioną regułą uczenia Widrowa-Hoffa – przyjmuje fundamentalną, wysoce zoptymalizowaną postać:

$$\Delta w_{ij} = -\eta \cdot \delta_i \cdot x_j \quad (12)$$

Gdzie η to narzucony krok uczenia (Learning Rate). Powyższe równanie tłumaczy nazwę algorytmu (propagacja wsteczna). Aby obliczyć uchyb w dowolnej warstwie głęboko ukrytej w strukturze CNN, sieć nie musi analizować całego obrazu od nowa. Zamiast tego, oblicza swój lokalny sygnał δ na podstawie sumy ważonej sygnałów δ propagowanych wstecznie z warstwy wyższej (znajdującej się bliżej klasyfikatora). Algorytm systematycznie cofa się od wyjścia, aż do samych filtrów spłotowych, gwarantując bezbłędne przypisanie wagom ich udziału w całkowitym błędzie predykcji.

2.6.4 Optymalizator AdamW

Samo wyliczenie gradientu nie aktualizuje wag, a klasyczne algorytmy (takie jak SGD) w trudnym krajobrazie funkcji straty często utykają w minimach lokalnych. W projekcie wykorzystano nowoczesny optymalizator **AdamW**, będący udoskonalonym wariantem algorytmu Adam. Jest on obecnie standardem w uczeniu sieci konwolucyjnych, gdyż poprawnie separuje mechanizm kary za wielkość wag (*Weight Decay*) od estymacji gradientu, co znacząco poprawia zdolność modelu do generalizacji.

Podstawowa mechanika aktualizacji wag w AdamW bazuje na tych samych zasadach co klasyczny Adam, wykorzystując adaptacyjne momenty pędu do sterowania procesem uczenia. W każdej iteracji t , aktualizacja konkretnej wagi bazuje na historii jej poprzednich gradientów g_t . Najpierw wyliczany jest *Pierwszy moment* (pęd – m_t) oraz *Drugi moment* (wariancja – v_t):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (13)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (14)$$

Gdzie hiperparametry β_1 i β_2 decydują o pojemności „pamięci” optymalizatora. Następnie, w celu uniknięcia obciążenia estymacji wynikającego z inicjalizacji zerami, stosuje się korektę (Bias Correction):

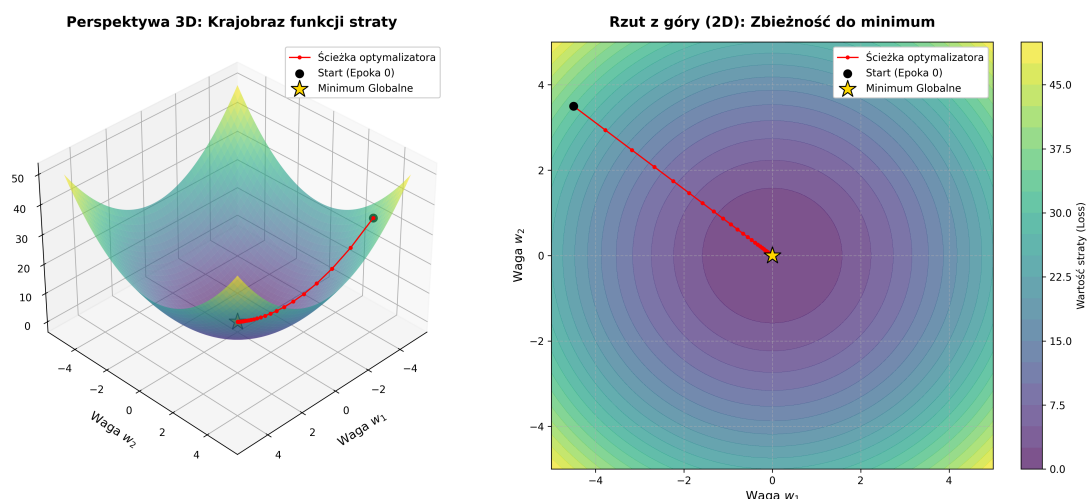
$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad (15)$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (16)$$

Ostateczna reguła aktualizacji wagi w optymalizatorze AdamW przyjmuje postać poprawioną o dekompozycję *Weight Decay*:

$$w_t = w_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda w_{t-1} \right) \quad (17)$$

Gdzie η to krok uczenia (Learning Rate), ϵ zapobiega dzieleniu przez zero, a λ to współczynnik kary za wagi, który w AdamW jest aplikowany bezpośrednio do parametru, a nie poprzez modyfikację gradientu. Takie podejście eliminuje wady klasycznego Adama (zbyt silną zależność kary od momentów) i zapewnia stabilną, szybką zbieżność omawianego modelu CNN.



Rysunek 9: Dwupanelowa wizualizacja przestrzeni błędów: perspektywa trójwymiarowa (lewa) oraz rzut topograficzny (prawa), obrazujące trajektorię zbieżności algorytmu optymalizacji do minimum globalnego.

2.6.5 Mechanika pętli uczącej i zarządzanie danymi

Przetworzenie całego zbioru danych naraz znacznie przekracza możliwości pamięciowe VRAM współczesnych akceleratorów graficznych. Dlatego proces treningu narzuca konieczność podzielenia danych na mniejsze, niezależne porcje, nazywane partiami (*Batch*). Zaktualizowanie wag na podstawie jednej partii (np. 128 obrazów) stanowi pojedynczy krok optymalizatora (iterację).

Gdy sieć przepuści przez algorytm propagacji wstecznej wszystkie partie składające się na zbiór treningowy, zamykana jest jedna epoka (*Epoch*). Proces ten jest powtarzany zadeklarowaną ilość razy. Na początku każdej epoki algorytmy ładujące (*DataLoaders*) dokonują przetasowania próbek (*Shuffle*), aby sieć nie nauczyła się sekwencyjności danych.

Aby zapobiec wąskiemu gardłom obliczeniowym (tzw. *Bottlenecks*), proces ładowania i augmentacji obrazów jest asynchroniczny. Wykorzystując wielowątkowość procesora (parametr *num_workers*), przygotowuje on w tle kolejne partie zdjęć, podczas gdy karta graficzna oblicza matematykę splotów dla obecnej partii. Po zakończeniu epoki treningowej, na modelu wywołany jest tryb ewaluacji, a zaktualizowane wagi testowane są na odseparowanym zbiorze walidacyjnym, co pozwala na rzetelną weryfikację zdolności generalizacyjnych.

2.7 Metryki ewaluacji i Macierz Pomyłek

Sama wartość funkcji straty (Loss) jest kluczowa dla optymalizatora, jednak z punktu widzenia inżyniera, nie jest ona miarą intuicyjną. Aby rzetelnie ocenić jakość wytrenowanego modelu, stosuje się ustandaryzowane metryki opierające się na analizie tzw. macierzy pomyłek (ang. *Confusion Matrix*).

Macierz pomyłek to struktura tabelaryczna, która zestawia rzeczywiste etykiety klas z predykcjami wygenerowanymi przez system. Dla pojedynczej, wybranej klasy (np. „Kamień” analizowany w systemie One-vs-Rest) predykcje dzieli się na cztery fundamentalne kategorie, co obrazuje Tabela 1:

		Wartość rzeczywista (Ground Truth)	
		Pozytywna (np. Kamień)	Negatywna (np. Inna)
Predykcja	Pozytywna	TP (Prawdziwie Pozytywna)	FP (Fałszywie Pozytywna)
Modelu	Negatywna	FN (Fałszywie Negatywna)	TN (Prawdziwie Negatywna)

Tabela 1: Teoretyczna struktura macierzy pomyłek z podziałem na klasy decyzyjne.

Wykorzystując wartości z poszczególnych komórek powyższej macierzy, wylicza się trzy główne metryki ewaluacyjne:

1. Dokładność / Precyzja (Precision)

Wskazuje, jak bardzo można ufać modelowi, gdy ten twierdzi, że wykrył daną klasę. Mierzy ona odsetek prawdziwie pozytywnych wyników wśród wszystkich, które model uznał za pozytywne:

$$Precision = \frac{TP}{TP + FP} \quad (18)$$

2. Czułość (Recall)

Określa, jaki procent wszystkich rzeczywistych obrazów z danej klasy został przez sieć pomyślnie odnaleziony i poprawnie sklasyfikowany.

$$Recall = \frac{TP}{TP + FN} \quad (19)$$

3. Miara F1 (F1-Score)

W idealnym scenariuszu model powinien mieć zarówno wysoką precyzję, jak i czułość. W praktyce często występuje między nimi kompromis (tzw. *trade-off*). Miara F1 to średnia harmoniczna obu tych wartości, pozwalająca na uzyskanie jednej, zbalansowanej oceny skuteczności modelu dla danej klasy:

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (20)$$

Wykorzystanie powyższych metryk, w połączeniu z globalnym wskaźnikiem poprawności całego systemu (*Accuracy*), pozwala na dokładną identyfikację słabych punktów architektury, takich jak faworyzowanie jednej z klas kosztem innych. Zestawienie tych wartości dla ostatecznej wersji mojego modelu zostało przedstawione w rozdziale poświęconym wynikom eksperymentów.

3 Przygotowanie i analiza danych

Jakość, struktura oraz sposób reprezentacji danych wejściowych mają fundamentalny wpływ na proces optymalizacji wag sieci neuronowej. W poniższym rozdziale zaprezentowano charakterystykę wykorzystanego zbioru obrazów oraz szczegółowo omówiono potok ich przetwarzania (ang. *Data Pipeline*), obejmujący normalizację matematyczną tensorów, asymetryczną augmentację oraz sprzętową optymalizację ładowania.

3.1 Charakterystyka i struktura zbioru danych

Fundamentem procesu uczenia sieci był zbiór danych typu *Rock-Paper-Scissors* pozyskany z platformy Kaggle. Wykorzystanie ustandaryzowanego, zewnętrznego zbioru zapewnia obiektywizm procesu treningowego i pozwala na weryfikację modelu w odniesieniu do uznanych benchmarków. Zbiór zawiera wyselekcjonowane obrazy dłoni o zróżnicowanej geometrii, reprezentujące trzy kluczowe klasy decyzyjne, których charakterystykę przedstawiono w Tabeli 2.

Klasa decyzyjna	Charakterystyka wizualna i wyzwania detekcji
Kamień (Rock)	Dłoń zwinięta w pięść. Główne wyzwanie stanowi brak wyraźnie wystających elementów geometrycznych (palców). Wymaga to od sieci skupienia się na ogólnej, zaokrąglonej bryle dłoni oraz analizie tekstury zgiętych paliczków.
Papier (Paper)	Dłoń otwarta, palce wyprostowane. Proces ekstrakcji cech opiera się na detekcji długich, pionowych krawędzi zewnętrznych oraz relacji przestrzennej między płaską powierzchnią dłoni a kciukiem.
Nożyce (Scissors)	Wyprostowany palec wskazujący i środkowy, reszta palców zwinięta. Klasa najbardziej złożona geometrycznie, wymagająca precyzyjnej detekcji ostrych kątów (układ typu „V”) oraz separacji tła między rozchylonymi palcami.

Tabela 2: Opis klas decyzyjnych w zbiorze wraz z analizą geometryczną detekcji.

Baza danych obejmuje łącznie 2188 obrazów. Ich dystrybucję klasową zestawiono poniżej:

- **Kamień:** 726 obrazów,
- **Papier:** 710 obrazów,
- **Nożyce:** 752 obrazy.

Wszystkie pliki wejściowe zostały zapisane w formacie PNG o rozdzielczości 300×200 pikseli. Zdjęcia zarejestrowano na jednolitym, zielonym tle z zachowaniem spójnych parametrów oświetlenia, co stanowi optymalną bazę dla wstępnego etapu szkolenia modelu CNN, ograniczając konieczność stosowania złożonych algorytmów segmentacji tła.

3.2 Podział zbioru

Aby model mógł zostać rzetelnie zweryfikowany, zgromadzona baza zdjęć musiała zostać podzielona na dwa niezależne zbiory: treningowy (stanowiący 80% danych) oraz walidacyjny (pozostałe 20%). Wymiary tych podzbiorów są wyliczane dynamicznie na podstawie całkowitej wielkości wczytanego zestawu (ang. *total_size*).

Podział ten z definicji musi być procesem losowym, aby sieć nie uczyła się sekwencyjności danych (np. że po pięciu „kamieniach” zawsze następuje „papier”). Jednocześnie musi on być bezwzględnie powtarzalny między kolejnymi uruchomieniami skryptu. W przeciwnym razie doszłoby do tzw. wycieku danych (ang. *Data Leakage*) – model w nowej sesji treningowej mógłby otrzymać w zbiorze uczącym te same zdjęcia, na których był wcześniej egzaminowany, co sztucznie zawyżyłoby ocenę jego skuteczności.

```
# DATASET SIZES:
total_size = len(full_train_dataset)
train_size = int(train_split * total_size)
val_size = total_size - train_size

# DIVISION:
indices = torch.randperm(total_size, generator=torch.Generator().manual_seed(179582)).tolist()
```

Rysunek 10: Dynamiczne wyliczanie rozmiarów zbiorów oraz mechanizm tasowania indeksów z wykorzystaniem stałego ziarna (seed) zapobiegającego wyciekom danych.

Jak ukazuje załączony listing, zbiór wszystkich dostępnych indeksów jest mieszany za pomocą funkcji `torch.randperm`, a następnie rozdzielany na pulę treningową i walidacyjną zgodnie z wcześniej wyliczonymi proporcjami. Wykorzystano tu dedykowany generator liczb pseudolosowych (`torch.Generator`), którego ziarno (ang. *seed*) zamrożono na unikalnej wartości numerycznej, odpowiadającej mojemu numerowi indeksu (179582). Gwarantuje to permanentną separację próbek walidacyjnych od środowiska treningowego, zapewniając pełną powtarzalność i wiarygodność prowadzonych w późniejszym etapie eksperymentów.

3.3 Wstępne przetwarzanie i normalizacja

Surowe obrazy cyfrowe o zróżnicowanych wymiarach nie mogą zostać bezpośrednio wprowadzone do wejścia sieci konwolucyjnej. W pierwszej fazie preprocessingu wszystkie klatki zostały ujednolicone do rozdzielczości 128×128 pikseli. Taki rozmiar stanowi optymalny kompromis pomiędzy zachowaniem wysokiej szczegółowości geometrycznej dłoni a akceptowalną złożonością obliczeniową.

Następnie obrazy przekształcono na struktury tensorowe. Operacja ta automatycznie rzuca dyskretnie wartości pikseli z przedziału $[0, 255]$ na zmiennoprzecinkowy przedział $[0, 1]$. Ostatnim, krytycznym krokiem z punktu widzenia matematyki uczenia maszynowego była **standaryzacja (normalizacja)** danych.

Zgodnie z implementacją w kodzie projektu, każdy z trzech kanałów barwnych c poddano transformacji wykorzystującej ustaloną średnią ($\mu = 0.5$) oraz odchylenie standardowe ($\sigma = 0.5$):

$$x_{norm}^{(c)} = \frac{x^{(c)} - \mu^{(c)}}{\sigma^{(c)}} = \frac{x^{(c)} - 0.5}{0.5} \quad (21)$$

Zabieg ten mapuje wszystkie sygnały wejściowe z przedziału $[0, 1]$ do ustandaryzowanego, symetrycznego przedziału $[-1, 1]$. Wyśrodkowanie danych wokół zera jest niezbędne dla optymalizatora gradientowego AdamW.

Gwarantuje to, że wagi początkowe sieci nie ulegną nasyceniu, zapobiega zjawisku zanikającego gradientu na funkcjach aktywacji i zapewnia stabilną zbieżność w początkowych epokach uczenia.

3.4 Proces augmentacji obrazów

Jednym z najważniejszych etapów logistyki danych było rygorystyczne rozdzielenie transformacji na potok treningowy i walidacyjny. Aby zrealizować to założenie, wykorzystano moduł `torchvision.transforms`. Pozwala on na łączenie wielu niezależnych operacji przetwarzania obrazu w jeden ustandaryzowany potok (za pomocą funkcji `Compose`). Dzięki takiemu podejściu zdefiniowane modyfikacje są aplikowane na zdjęciach dynamicznie (w czasie rzeczywistym), bezpośrednio przed przekazaniem ich do sieci.

```
# IMAGE PROCESSING FOR TRAINING
train_transform = transforms.Compose([
    transforms.Resize((img_size, img_size)),

    transforms.RandomRotation(degrees=90),
    transforms.RandomAffine(degrees=0, translate=(0.2, 0.2), scale=(0.7, 1.3)),
    transforms.ColorJitter(brightness=0.4, contrast=0.4, saturation=0.4, hue=0.2),

    transforms.ToTensor(),

    transforms.RandomErasing(p=0.2, scale=(0.02, 0.15)),

    transforms.Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5])
])

# IMAGE PROCESSING FOR VALIDATION
val_transform = transforms.Compose([
    transforms.Resize((img_size, img_size)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5])
])
```

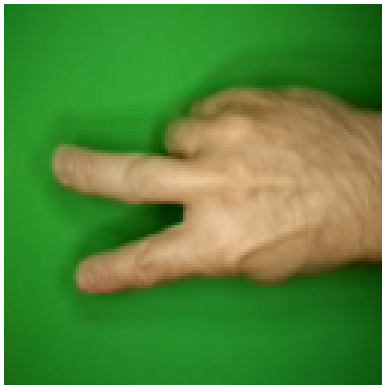
Rysunek 11: Kod źródłowy implementujący kompozycję transformacji dla zbioru treningowego oraz walidacyjnego.

Jak widać na załączonym listingu, środowisko walidacyjne (`val_transform`) pozostało w formie nienaruszonej (zastosowano na nim wyłącznie opisaną wyżej zmianę rozmiaru i normalizację), pełniąc rolę obiektywnego, „czystego” egzaminatora modelu. Z kolei na zbiorze treningowym (`train_transform`) zastosowano agresywną augmentację danych (*Data Augmentation*), wprowadzającą sztuczny chaos do obrazów:

- **Random Rotation:** Ekstremalne obroty obrazu w zakresie od -90 do 90 stopni. Wymuszają na sieci naukę niezmienniczości na rotację (rozpoznawanie dłoni niezależnie od kąta jej ułożenia).
- **Random Affine:** Transformacje afiniczne, w tym swobodne skalowanie (zoom) oraz przesunięcia w osiach X i Y, zapobiegające zapamiętaniu przez sieć centralnego ułożenia dłoni.
- **Color Jitter:** Losowe modyfikacje jasności, kontrastu, saturacji oraz barwy. Transformacja ta skutecznie uniezależnia model od specyficznych warunków oświetleniowych panujących na zdjęciach treningowych.
- **Random Erasing:** Zaawansowana technika zamazująca losowe, prostokątne fragmenty tensora. Algorytm wymusza w ten sposób klasyfikację na podstawie globalnego kształtu i zmusza do pracy wszystkie filtry spłotowe, uniemożliwiając poleganie na jednym, specyficznym pikselu.

Wykorzystanie tak skonstruowanego, asymetrycznego potoku danych skutecznie blokuje zjawisko przeuczenia (Overfitting), wymuszając na sieci budowę głębokich i odpornych na szum reprezentacji cech.

Aby udowodnić teoretyczny wpływ zaimplementowanych transformacji na mechanikę ekstrakcji cech, przeprowadzono badanie porównawcze. Wygenerowano mapy aktywacji w warstwach spłotowych dla surowego obrazu referencyjnego (wyidealizowanego) oraz dla tego samego obrazu poddanego skrajnym deformacjom wynikającym z aktywnego potoku augmentacji danych.



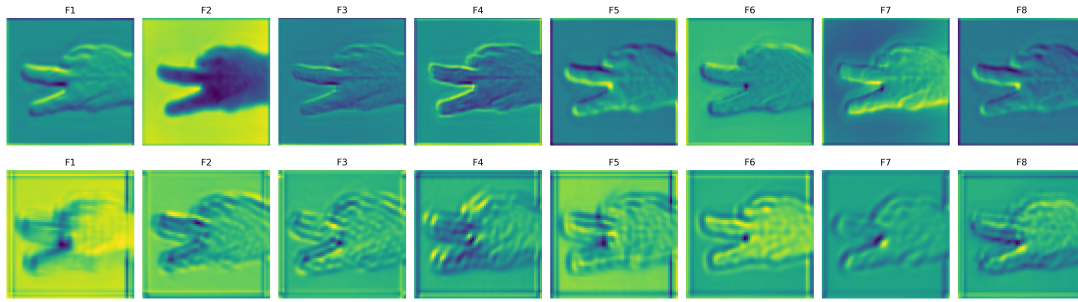
(a) Oryginalny obraz referencyjny



(b) Obraz po zastosowaniu augmentacji

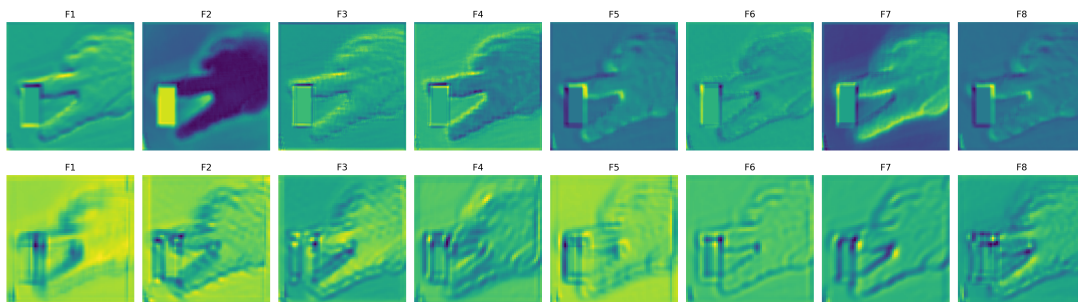
Rysunek 12: Wizualizacja deformacji wejściowego tensora pod wpływem potoku treningowego (m.in. *Random Rotation*, *Color Jitter* oraz *Random Erasing*).

Rysunek 12 (b) w sposób bezpośredni ilustruje stopień degradacji przestrzennej i barwnej, z jakim musi mierzyć się klasyfikator podczas pojedynczej iteracji uczącej. Znaczące rotacje oraz nałożony szum (w postaci utraconych fragmentów danych) stanowią rygorystyczny test dla architektury.



Rysunek 13: Analiza obrazu wyidealizowanego (bez augmentacji). Od góry: mapy aktywacji z warstwy płytkiej (`conv1`) oraz z warstwy głębokiej (`conv2`). Widoczna jest łatwa izolacja krawędzi od jednolitego, zielonego tła.

Powyższa wizualizacja dowodzi, że w przypadku czystego, wyidealizowanego obrazu, sieć niemal natychmiast, z dużą pewnością, izoluje obiekt z jednorodnego otoczenia. Choć skutkuje to mocnymi aktywacjami w warstwach głębokich, stwarza ogromne ryzyko zapamiętania specyfiki tła (co wyraźnie widać na filtrach maskujących warstwę `conv1`), co w warunkach rzeczywistych prowadziłoby do błędów klasyfikacji.



Rysunek 14: Reakcja wag sieci na zaburzenia wywołane augmentacją (odpowiednio: warstwa `conv1` oraz `conv2`). Zniekształcenia wymuszają na filtrach analizę wysoce abstrakcyjnej topologii dłoni zamiast polegania na prostych krawędziach tła.

Zastosowanie augmentacji całkowicie zaburza proces prostej ekstrakcji krawędzi – na mapach aktywacyjnych widoczny jest szum przestrzenny i zakłócenia wynikające z wprowadzonych rotacji i ubytków (*Random Erasing*). Mechanizm ten fizycznie wymusza na wagach sieci znacznie cięższą pracę obliczeniową. Zmusza algorytm do zignorowania rozmytych barw na rzecz skupienia się na uniwersalnych i niezmiennych cechach topologicznych gestu (takich jak struktura palców i ich wzajemne położenie). Przedstawione mapy stanowią twardy, matematyczny dowód na efektywność zaimplementowanych zabezpieczeń regularyzacyjnych przed zjawiskiem *overfittingu*.

3.5 Mechanizmy ładowania zbiorów

Odpowiednie przygotowanie macierzy w pamięci masowej to zaledwie połowa sukcesu. Głównym wąskim gardłem (ang. *bottleneck*) w architekturach uczenia głębokiego rzadko jest sama karta graficzna (GPU). Znacznie częściej problemem jest procesor (CPU) oraz dysk, które nie nadążają z dekodowaniem i przesyłaniem kolejnych pakietów zdjęć.

Aby zapobiec przestojom akceleratora obliczeniowego, w pliku `data_pipeline.py` zaimplementowano wysoce zoptymalizowany system zasilania sieci oparty na klasie `DataLoader` z biblioteki PyTorch.

```
# FINAL DATASETS
train_dataset = torch.utils.data.Subset(full_train_dataset, train_idx)
val_dataset = torch.utils.data.Subset(full_val_dataset, val_idx)
# LOADING:
train_loader = DataLoader(
    train_dataset, batch_size=batch_size, shuffle=True,
    num_workers=4, pin_memory=True, persistent_workers=True
)
val_loader = DataLoader(
    val_dataset, batch_size=batch_size, shuffle=False,
    num_workers=4, pin_memory=True, persistent_workers=True
)
```

Rysunek 15: Implementacja mechanizmów asynchronicznego ładowania danych i optymalizacji pamięci.

Jak ukazuje załączony listing, obiekty ładujące dla zbioru treningowego i walidacyjnego różnią się kluczowym parametrem tasowania, współdzieląc jednocześnie zaawansowane ustawienia optymalizacji sprzętowej:

- **Tasowanie próbek (`shuffle`):** Dla zbioru treningowego parametr ten ustawiono na `True`. Jest to wymóg krytyczny, zapobiegający wyuczeniu się przez sieć sekwencyjności danych (zapamiętaniu wzorców ułożenia obrazów w folderze). Z kolei w potoku walidacyjnym (`shuffle=False`) tasowanie jest wyłączone – kolejność próbek na etapie obiektywnego „egzaminu” nie wpływa na wartość metryk, a wyłączenie tej operacji oszczędza cenne zasoby procesora.
- **Wielowątkowość (`num_workers=4`):** System operacyjny deleguje proces ładowania obrazów i obliczania na nich augmentacji matematycznych do czterech niezależnych procesów potomnych działających w tle. Kiedy GPU przetwarza obecną partię danych (Batch), CPU asynchronicznie przygotowuje już partię następną, minimalizując czasy bezczynności (tzw. *GPU Starvation*).
- **Blokowanie stron pamięci (`pin_memory=True`):** Parametr ten wymusza zaalokowanie specjalnego obszaru pamięci operacyjnej RAM (tzw. *page-locked memory*), który nie podlega stronicowaniu na dysk. Dzięki temu transfer gigabajtów danych macierzowych bezpośrednio do pamięci VRAM karty graficznej przez magistralę PCIe odbywa się z wykorzystaniem bezpośredniego dostępu do pamięci (DMA), co drastycznie zwiększa przepustowość.
- **Utrzymanie procesów (`persistent_workers=True`):** Zaawansowana optymalizacja cyklu życia wątków. Domyślnie PyTorch zamyka procesy ładujące po zakończeniu każdej epoki i tworzy je na nowo w kolejnej, co generuje ogromny narzut czasowy (tzw. *overhead*). Utrzymanie aktywnych procesów w tle eliminuje to opóźnienie, gwarantując płynne i błyskawiczne przejścia między kolejnymi iteracjami uczącymi.

4 Architektura i implementacja modelu CNN

Rozdział ten poświęcony jest szczegółowej analizie technicznej autorskiego modelu `RPS_CNN`. Przedstawiono tu nie tylko samą implementację warstw, ale także uzasadnienie matematyczne i inżynierskie zastosowanych rozwiązań, które odpowiadają za wysoką skuteczność klasyfikacji gestów dłoni.

4.1 Struktura opracowanej sieci CNN

Architektura modelu `RPS_CNN` została zorganizowana w hierarchiczny sposób, podzielony na dwa bloki ekstrakcji cech oraz moduł klasyfikacyjny. Taka separacja pozwala na sukcesywne przechodzenie od analizy niskopoziomowej (wykrywanie krawędzi) do analizy wysokopoziomowej (rozpoznawanie złożonych kształtów gestów).

Warstwa	Typ	Tensor wyj.	Opis operacji
Input	-	$3 \times 128 \times 128$	Obraz RGB znormalizowany.
conv1	Conv2d	$N \times 128 \times 128$	Splot $K \times K$, padding='same'.
silu1	SiLU	$N \times 128 \times 128$	Nieliniowość aktywacyjna.
pool1	MaxPool2d	$N \times 64 \times 64$	Redukcja przestrzenna 2×2 .
conv2	Conv2d	$2N \times 64 \times 64$	Splot, podwojenie liczby filtrów ($2N$).
silu2	SiLU	$2N \times 64 \times 64$	Nieliniowość aktywacyjna.
pool2	MaxPool2d	$2N \times 32 \times 32$	Redukcja przestrzenna 2×2 .
flatten	Flatten	$32 \times 32 \times 2N$	Wektoryzacja tensora.
fc1	Linear	64	Pierwsza warstwa gęsta (Dense).
silu3	SiLU	64	Nieliniowość.
dropout	Dropout	64	Regularyzacja (p=0.2).
fc2	Linear	3	Warstwa wyjściowa (klasy).

Tabela 3: Szczegółowa architektura sieci. N oznacza liczbę filtrów (`num_filters`), K rozmiar jądra (`kernel_size`).

4.2 Parametryzacja architektury w konstruktorze (`__init__`)

Sercem układu decyzyjnego jest klasa `RPS_CNN`, której konstruktor został zaprojektowany w sposób elastyczny. Zamiast operować na sztywno zakodowanych wartościach, sieć przyjmuje parametry konfiguracyjne, takie jak `kernel_size` czy `num_filters`, co pozwala na jej szybką adaptację w ramach późniejszych badań eksperymentalnych.

```

class RPS_CNN(nn.Module):
    def __init__(self, kernel_size=3, num_filters=16, hidden_neurons=64, dropout_rate=0.2):
        super(RPS_CNN, self).__init__()

        #Zmienne wyciągnięte do eksperymentów badawczych
        self.kernel_size = kernel_size
        self.num_filters = num_filters
        self.hidden_neurons = hidden_neurons
        self.dropout_rate = dropout_rate

        #-> WARSTWA KONWOLUCYJNA <-
        # padding='same' zachowuje rozmiar krawędzi obrazu
        self.conv1 = nn.Conv2d(in_channels=3, out_channels=self.num_filters, kernel_size=self.
kernel_size, padding='same')
        # Funkcja aktywacji SiLU (zamiast ReLU) - lepsze wyniki w widzeniu maszynowym
        self.silu1 = nn.SiLU()
        # MaxPool zmniejsza rozdzielczość o połowę (ze 128x128 na 64x64)
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)

        #-> WARSTWA KONWOLUCYJNA <-
        # (Podwajamy liczbę filtrów)
        self.conv2 = nn.Conv2d(in_channels=self.num_filters, out_channels=self.num_filters * 2
, kernel_size=self.kernel_size, padding='same')
        self.silu2 = nn.SiLU()
        # MaxPool znów zmniejsza rozdzielczość (z 64x64 na 32x32)
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)

        #-> KLASYFIKATOR (Warstwy w pętli potężone) <-
        # Obliczamy ile pikseli wyjdzie z ostatniego basenu (32 * 32 * liczba kanałów)
        flattened_size = 32 * 32 * (self.num_filters * 2)

        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(flattened_size, self.hidden_neurons)
        self.silu3 = nn.SiLU()

        # DROPOUT
        self.dropout = nn.Dropout(self.dropout_rate)
        # Zwraca wektor 3 prawdop. (kamień, papier, nożyce)
        self.fc2 = nn.Linear(self.hidden_neurons, 3)

```

Rysunek 16: Implementacja konstruktora sieci z parametryzacją warstw konwolucyjnych.

Kluczowe decyzje projektowe widoczne na poziomie inicjalizacji to:

- **Padding='same'**: Zastosowanie tego parametru w warstwach Conv2d jest krytyczne – pozwala zachować spójność przestrzenną obrazu. Dzięki temu każda kolejna warstwa splotowa operuje na pełnym polu widzenia, co minimalizuje zjawisko utraty cech na brzegach kadru.
- **Funkcja aktywacji SiLU**: W przeciwieństwie do tradycyjnej funkcji ReLU, SiLU posiada łagodniejszą krzywą w obszarze wartości ujemnych ($x \cdot \sigma(x)$). Dzięki temu sieć jest w stanie przepuszczać ujemne wartości gradientu, co znacząco poprawia dynamikę uczenia i zapobiega efektowi "martwych neuronów".
- **Automatyczne wyliczanie Flattened Size**: Obliczenie `flattened_size = 32 * 32 * (self.num_filters * 2)` dowodzi pełnej kontroli nad wymiarowością tensora. Po przejściu przez dwa bloki MaxPool2d (każdy redukujący wymiar przestrzenny o połowę), wejściowa rozdzielczość 128×128 zostaje bezbłędnie sprowadzona do postaci 32×32 , uniezależniając architekturę od ręcznych przeliczeń.

4.3 Przepływ danych i hierarchia cech (Forward Pass)

Metoda `forward` definiuje ścieżkę krytyczną, przez którą przechodzi tensor danych – począwszy od cyfrowej reprezentacji surowych pikseli, aż do wygenerowania ostatecznej predykcji.

```
def forward(self, x):  
    # Forward Pass  
    x = self.pool1(self.silu1(self.conv1(x)))  
    x = self.pool2(self.silu2(self.conv2(x)))  
    x = self.flatten(x)  
    x = self.dropout(self.silu3(self.fc1(x)))  
    x = self.fc2(x)  
    return x
```

Rysunek 17: Logika przepływu sygnału (Forward Pass) w klasie `RPS_CNN`.

Działanie tej metody zrealizowano w pięciu ściśle określonych krokach matematycznych:

1. **Blok Splotowy I (Ekstrakcja cech):** Dane wejściowe trafiają do warstwy `conv1`, gdzie filtr o rozmiarze $K \times K$ dokonuje ekstrakcji niskopoziomowych cech wizualnych (krawędzie, gradienty światła). Zastosowana funkcja aktywacji `SiLU` narzuca systemowi nieliniowość, a warstwa `pool1` redukuje szum informacyjny i zmniejsza wymiarowość przestrzenną.
2. **Blok Splotowy II (Hierarchia cech):** Sygnał przechodzi przez `conv2`, gdzie zdefiniowana wcześniej liczba filtrów zostaje podwojona ($N \rightarrow 2N$). Jest to kluczowe dla zwiększenia gęstości abstrakcji – sieć łączy wykryte proste krawędzie w złożone struktury (np. zarys dłoni). Zastosowanie warstwy `pool2` zapewnia systemowi uodpornienie na przesunięcia przestrzenne (*translational invariance*).
3. **Wektoryzacja:** Warstwa `flatten` przekształca trójwymiarowy tensor cech na zredukowany, jednowymiarowy wektor, stanowiąc matematyczny pomost dla klasyfikatora gęstego.
4. **Klasyfikacja gęsta (Wnioskowanie):** Wektor trafia do warstwy `fc1`. Aplikacja warstwy `Dropout` (20%) pełni rolę kluczowego zabezpieczenia przed przeuczeniem (*overfitting*) – sieć zmuszana jest do nauki rozproszonych reprezentacji, uniemożliwiając mechaniczne zapamiętanie danych treningowych.
5. **Decyzja Wyjściowa:** Ostatnia warstwa liniowa (`fc2`) redukuje przetworzony wektor do 3 logitów, reprezentujących surowe wagi przypisane klasom: kamień, papier lub nożyce.

4.4 Pętla ucząca i zarządzanie optymalizacją (Training Loop)

Zdefiniowanie topologii sieci nie rozwiązuje problemu detekcji – model musi przejść przez proces iteracyjnej aktualizacji wag. Logika tego procesu została odseparowana od definicji strukturalnej i zaimplementowana w głównym skrypcie sterującym `train.py`.

Pętla ucząca (*Training Loop*) została zbudowana w oparciu o współpracujące ze sobą komponenty matematyczne, których inicjalizację przedstawiono na Rysunku 18.

```
def train_and_evaluate(
    batch_size=128,
    lr=0.001,
    epochs=20,
    kernel_size=3,
    num_filters=16,
    dropout_rate=0.2,
    plot_results=False,
    save_model=False
):
    filename = f"Batch_{batch_size}_LR_{lr}_Kernel_{kernel_size}_Filters_{num_filters}_Dropout_{dropout_rate}"
    print("\n--- Start Treningu | "+filename.replace("_", " ")+" ---")
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    # 1. Tworzymy model przepuszczając parametry z funkcji
    model = RPS_CNN(
        kernel_size=kernel_size,
        num_filters=num_filters,
        dropout_rate=dropout_rate
    ).to(device)

    # 2. Ładujemy dane przepuszczając batch_size
    train_loader, val_loader, classes = get_data_loaders(batch_size=batch_size)

    # 3. Sędzia i Nauczyciel
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.AdamW(model.parameters(), lr=lr)

    # Krok schedulera dostosowany do liczby epok
    scheduler = lr_scheduler.CosineAnnealingLR(optimizer, T_max=epochs, eta_min=lr/1000)

    # --- LISTY NA DANE DO WYKRESÓW ---
    train_losses, val_losses = [], []
    train_accuracies, val_accuracies = [], []

    best_val_acc = 0.0

    # --- EARLY STOPPING VARIABLES ---
    patience = 10
    epochs_no_improve = 0
    min_delta = 0.001
```

Rysunek 18: Inicjalizacja optymalizatora, funkcji straty oraz harmonogramu uczenia.

Wyróżnić tu należy:

- **Kryterium oceny (Funkcja Straty):** Wykorzystano Kategorię Entropię Krzyżową (`nn.CrossEntropyLoss`), która wylicza uchyb poprzez rygorystyczne karanie modelu za rozbieżności między predykcją a rzeczywistą etykietą obrazu.
- **Nauczyciel (Optymalizator):** Zastosowano zaawansowany algorytm AdamW. Wylicza on adaptacyjne momenty pędu niezależnie dla każdej wagi, wspierając się poprawnie odseparowaną dekompozycją kary za ich wielkość (*Weight Decay*).

Przepływ danych w pojedynczej epoce treningowej dzieli się na dwie rygorystyczne fazy: trening (z modyfikacją wag) oraz ewaluację (symulującą środowisko produkcyjne).

Faza 1: Trening i Propagacja Wsteczna

Po przełączeniu modelu w tryb nauki (`model.train()`), aktywowane zostają mechanizmy regularyzacyjne, takie jak Dropout. Dla każdej przetworzonej partii obrazów (*batch*), algorytm realizuje cykl fundamentalnych operacji ukazany na Rysunku 19.

```
for epoch in range(epochs):

    # --- FAZA 1: TRENING ---
    model.train()
    running_train_loss = 0.0
    correct_train = 0
    total_train = 0

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        predictions = model(images)
        loss = criterion(predictions, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_train_loss += loss.item()

        _, predicted_class = torch.max(predictions, 1)
        total_train += labels.size(0)
        correct_train += (predicted_class == labels).sum().item()

    avg_train_loss = running_train_loss / len(train_loader)
    train_acc = 100 * correct_train / total_train

    train_losses.append(avg_train_loss)
    train_accuracies.append(train_acc)
```

Rysunek 19: Realizacja fazy treningowej z mechanizmem propagacji wstecznej.

1. **Forward Pass:** Tensor obrazów transferowany jest do pamięci VRAM akceleratora, a następnie przetwarzany przez graf obliczeniowy modelu w celu uzyskania wektora predykcji (`predictions`).
2. **Loss Calculation:** Funkcja straty oblicza wartość błędu w danej iteracji.
3. **Zero Gradients:** Wywołanie `optimizer.zero_grad()` czyści bufor gradientowe. Jest to krok krytyczny zapobiegający niepożądanemu akumulacji pochodnych z poprzednich paczek danych.

4. **Backward Pass:** Metoda `loss.backward()` uruchamia silnik Autograd, który wylicza pochodne cząstkowe błędu względem wszystkich parametrów sieci.
5. **Optimizer Step:** Metoda `optimizer.step()` aplikuje wyliczone gradienty do struktury wagowej, minimalizując funkcję straty.

Faza 2: Ewaluacja i Blokada Gradientów

Po przetworzeniu wszystkich danych z zestawu uczącego, sieć przechodzi w tryb walidacji (`model.eval()`), co ilustruje Rysunek 20.

```
# --- FAZA 2: WALIDACJA (EGZAMIN) ---
model.eval()
running_val_loss = 0.0
correct_val = 0
total_val = 0

with torch.no_grad():
    for images, labels in val_loader:
        images, labels = images.to(device), labels.to(device)
        predictions = model(images)

        loss = criterion(predictions, labels)
        running_val_loss += loss.item()

        _, predicted_class = torch.max(predictions, 1)
        total_val += labels.size(0)
        correct_val += (predicted_class == labels).sum().item()

    avg_val_loss = running_val_loss / len(val_loader)
    val_acc = 100 * correct_val / total_val

    val_losses.append(avg_val_loss)
    val accuracies.append(val_acc)

    scheduler.step()

# RAPORT DO KONSOLI
current_lr = scheduler.get_last_lr()[0]
print(f"Epoka [{epoch+1}/{epochs}]\t Train Acc: {train_acc:.2f}% | Val Acc: {val_acc:.2f}% | LR: {current_lr:.6f}")

# --- ZAPISYWANIE NAJLEPSZEGO MODELU I EARLY STOPPING ---
if val_acc > best_val_acc + min_delta:
    best_val_acc = val_acc
    epochs_no_improve = 0 # Resetujemy licznik, bo jest poprawa!

    if save_model:
        print(f"->\t[UWAGA: Nowy rekord walidacji! Zapisuje model ({best_val_acc:.2f}%)]\t<-")
        torch.save(model.state_dict(), 'rps_model_best.pth')
else:
    epochs_no_improve += 1
    print(f"->\t[Brak poprawy: {epochs_no_improve}/{patience}]\t<-")

    if epochs_no_improve >= patience:
        print(f"!!! Early Stopping: Model przestał się poprawiać po {epoch+1} epokach. !!!")
        break
```

Rysunek 20: Faza walidacji realizowana w izolacji od grafu obliczeniowego.

Cały blok ewaluacyjny otoczono kontekstem `with torch.no_grad()`. Deklaracja ta całkowicie wyłącza śledzenie operacji przez silnik różniczkowania. Oznacza to drastyczne zmniejszenie alokacji pamięci VRAM oraz przyspieszenie wnioskowania, gwarantując jednocześnie rygor badawczy – system w żaden sposób nie "uczy się" na zestawie walidacyjnym, służącym wyłącznie jako miara jakości uogólniania wiedzy.

4.5 Adaptacyjne harmonogramowanie kroku uczenia (Cosine Annealing)

Zastosowanie stałego współczynnika uczenia (*Learning Rate* – LR) jest w nowoczesnych systemach głębokich uważane za podejście suboptymalne. Zbyt wysoki krok na końcu treningu zapobiega osadzeniu się modelu w globalnym minimum, natomiast zbyt mały krok na początku prowadzi do ekstremalnie powolnej konwergencji.

W rozwiązaniu zaimplementowano harmonogramowanie kroku uczenia metodą **Cosine Annealing** (`lr_scheduler.CosineAnnealingLR`). Algorytm ten dynamicznie modyfikuje wartość *Learning Rate* po zakończeniu każdej epoki, zgodnie z przebiegiem połówki funkcji cosinusoidalnej: rozpoczyna trening od zadeklarowanej wartości maksymalnej, a w miarę zbliżania się do limitu epok (`T_max`), płynnie wygasza krok uczenia. Gwarantuje to wysoce precyzyjne osadzenie się wag w dnie funkcji straty na ostatnich etapach uczenia.

4.6 Mechanizm Checkpointingu i Ewaluacja Ciągła

Wielogodzinny proces optymalizacji struktury neuronowej obarczony jest ryzykiem spadku generalizacji wiedzy na rzecz perfekcyjnego zapamiętywania zbioru treningowego. Zjawisko to powoduje, że wagi wygenerowane w ostatniej epoce uczenia rzadko stanowią optymalne rozwiązanie.

Zabezpieczeniem wdrożonym w układzie sterującym jest mechanizm izolowanego *Checkpointingu*. Podczas fazy walidacyjnej algorytm oblicza rygorystyczną skuteczność klasyfikacji (*Validation Accuracy*) i na bieżąco porównuje ją z dotychczasowym maksimum (`best_val_acc`). W przypadku wykrycia konfiguracji o niższym uchybie generalizacyjnym, następuje natychmiastowy zrzut matrycy wag do pliku binarnego (`rps_model_best.pth`). Technika ta pozwala na bezstratne odzyskanie optymalnego stanu modelu bez względu na ewentualne procesy degradacji (przeuczenia) występujące w końcowych fazach działania skryptu.

5 Badania eksperymentalne i optymalizacja hiperparametrów

Głównym celem badań eksperymentalnych było wyznaczenie optymalnej przestrzeni hiperparametrów dla opracowanej architektury RPS_CNN, gwarantującej najwyższą skuteczność generalizacji przy jednoczesnym zachowaniu stabilności numerycznej. Proces badawczy zrealizowano z wykorzystaniem paradygmatu przeszukiwania siatki (*Grid Search*), w którym ewaluacji poddano kilkadziesiąt niezależnych konfiguracji wagowych.

5.1 Metodologia badań i środowisko testowe

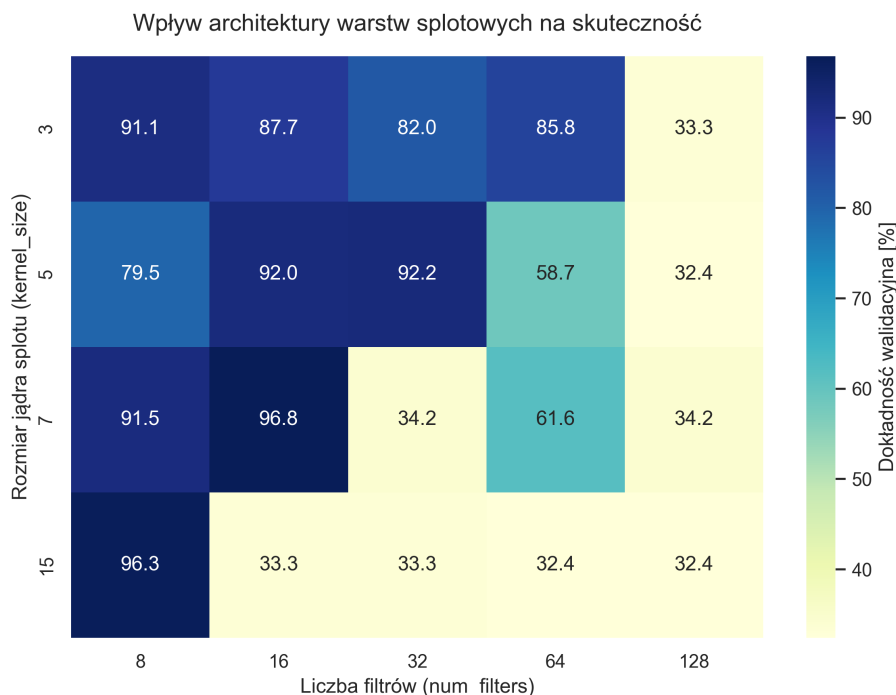
Aby zagwarantować pełną powtarzalność i rygor metodologiczny, eksperymenty przeprowadzono w ustandaryzowanych warunkach. Każdy model był inicjalizowany z wykorzystaniem tego samego ziarna losowości, a proces uczenia podlegał ścisłej kontroli mechanizmów nadzorujących:

- **Walidacja krzyżowa (Hold-out):** Dataset został rygorystycznie podzielony na zbiór treningowy (80%) i walidacyjny (20%). Skuteczność uogólniania wiedzy (*Validation Accuracy*) mierzono wyłącznie na danych, których model nie aktualizował w procesie wstecznej propagacji błędu.
- **Limit epok i Early Stopping:** Zamiast arbitralnego zamrażania liczby epok, w fazie przeszukiwania siatki wprowadzono górny limit wynoszący 50 epok oraz mechanizm wczesnego zatrzymywania (*Early Stopping*) z parametrem cierpliwości (*patience* = 10). Jeśli skuteczność walidacyjna nie uległa poprawie o minimum 0.1% przez 10 kolejnych iteracji, trening był automatycznie przerywany. Algorytm ten znacząco zredukował czas obliczeniowy, błyskawicznie odrzucając konfiguracje ulegające dywergencji.
- **Izolacja zmiennych:** Badania podzielono na trzy sekwencyjne etapy. W każdym etapie modyfikowano wyłącznie wybraną podgrupę hiperparametrów, zamrażając pozostałe zmienne na wartościach uznanych za optymalne w krokach poprzednich.

Oceny jakości modeli dokonywano na podstawie dwóch głównych metryk: końcowej skuteczności na zbiorze walidacyjnym (*Val_Accuracy*) oraz luki uogólniania, będącej różnicą między predykcją na zbiorze treningowym a testowym.

5.2 Etap I: Architektura przestrzenna i ekstrakcja cech

Pierwszy etap badań miał na celu wyłonienie optymalnej topologii warstw spłotowych. Analizie poddano zależność między wielkością pola recepcyjnego (rozmiar jądra `kernel_size`) a pojemnością informacyjną bloku (liczba wyodrębnianych map cech `num_filters`).



Rysunek 21: Mapa ciepła skuteczności walidacyjnej w funkcji rozmiaru jądra spłotu i liczby filtrów.

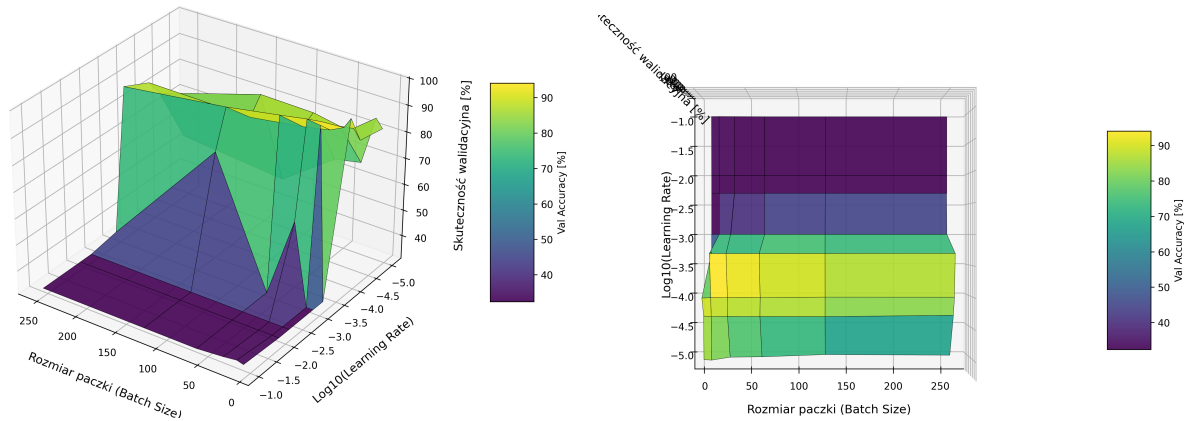
Wnioski płynące z analizy przestrzeni wyników są jednoznaczne. Zaobserwowano, że drastyczne zwiększanie rozmiaru jądra (powyżej 7×7) w połączeniu z dużą liczbą filtrów (np. 128) prowadzi do całkowitej degradacji modelu (wyniki rzędu 32-34%, co odpowiada progowi losowego zgadywania klasy). Jest to zjawisko typowe dla sieci, które tracą zdolność do wykrywania drobnych krawędzi na rzecz zbyt ogólnego "rozmywania" obrazu w początkowych fazach spłotu.

Najwyższą skuteczność generalizacji (96.8%) osiągnęła konfiguracja z parametrami `kernel_size = 7` oraz `num_filters = 16`. Parametry te zostały uznane za globalne optimum architektoniczne i zamrożone dla kolejnych etapów badań.

5.3 Etap II: Dynamika uczenia i stabilność gradientowa

Najbardziej wrażliwym elementem strojenia głębokich sieci neuronowych jest balans między rozmiarem paczki danych (`batch_size`) a współczynnikiem uczenia optymalizatora (`learning_rate`). W Etapie II zbadano dynamikę zbieżności modelu przy różnych kombinacjach tych wartości.

W celu pełnego zobrazowania ukształtowania powierzchni błędu i skuteczności w wielowymiarowej przestrzeni parametrów optymalizacyjnych, wyniki rzutowano na dwukierunkowe wykresy trójwymiarowe, zaprezentowane na Rysunku 22.



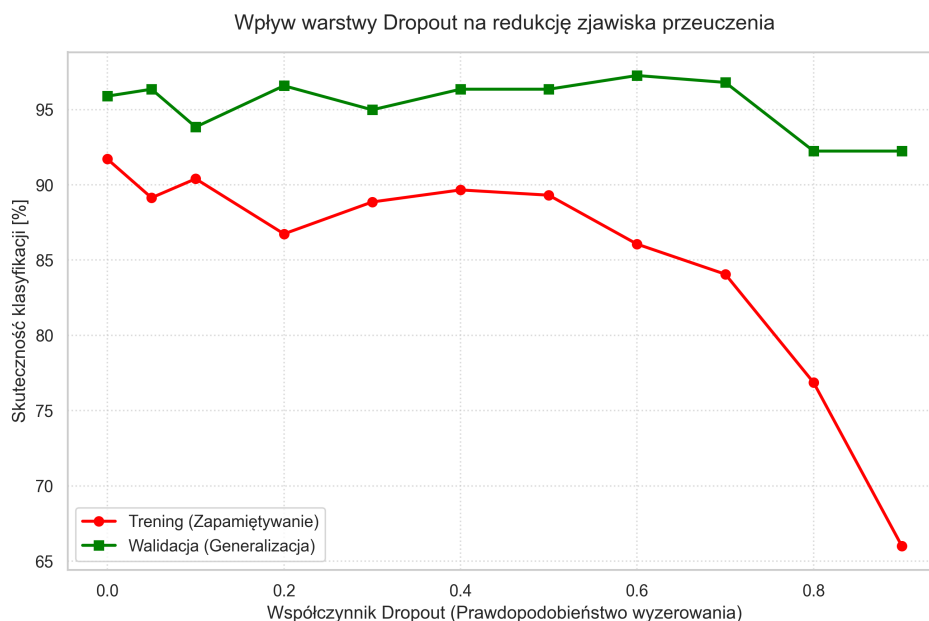
Rysunek 22: Topologia zbieżności i stabilności numerycznej optymalizatora AdamW w przestrzeni hiperparametrów.

Wizualizacje wyraźnie ujawniają obecność rozległej strefy niestabilności numerycznej dla dużych kroków uczenia ($\eta \geq 0.01$). W tym regionie model wykazuje całkowitą dywergencję gradientu, niezależnie od przyjętego rozmiaru batcha. Silnik optymalizatora AdamW drastycznie nadpisuje korekty wag, oscylując wokół wartości skuteczności 33.3%, co odpowiada prawdopodobieństwu zdarzenia losowego dla trzech klas.

Właściwy punkt stabilności ujawnia się w dolnej części spektrum (Rys. 22). Przy ograniczeniu budżetu treningowego do 50 epok, najwyższą lokalną skuteczność osiągnięto dla zestawienia `batch_size` = 16 oraz `lr` = 0.0005, dającego wynik 98.63%. Częstsza aktualizacja wag przy mniejszej paczce pozwoliła sieci na szybszą adaptację w warunkach narzuconego limitu iteracji.

5.4 Etap III: Regularyzacja i ochrona przed przeuczeniem

Ostatni etap walidacji środowiska dedykowano analizie warstwy **Dropout** – mechanizmowi celowego wyłączania ułamka neuronów w warstwie w pełni połączonej, co ma zapobiegać zjawisku nadmiernego dopasowania do zbioru treningowego (*overfitting*). Rozkład skuteczności predykcyjnej przedstawia Rysunek 23.



Rysunek 23: Luka uogólniania między zbiorem treningowym a walidacyjnym w funkcji parametru Dropout.

Badanie potwierdziło teoretyczne założenia regularyzacji. Przy całkowicie wyłączonym mechanizmie (**Dropout** = 0.0) widoczna jest asymetria między zestawem treningowym a walidacyjnym. Wraz ze wzrostem parametru w przedziale 0.1 – 0.4, tak zwana "luka uogólniania" (*generalization gap*) drastycznie maleje, co świadczy o rosnącej odporności sieci na szum wizualny. Optymalnym kompromisem okazała się wartość 0.2, powyżej której (od 0.6 wzwyż) sieć zaczyna cierpieć z powodu destrukcji zbyt dużej ilości informacji płynącej do klasyfikatora.

5.5 Weryfikacja końcowa i konfiguracja modelu optymalnego

Zwieńczeniem procesu badawczego było przeprowadzenie długofalowej sesji treningowej łączącej optymalne punkty wyznaczone we wszystkich trzech etapach cząstkowych. W oparciu o zebrane wnioski numeryczne sformułowano ostateczny, produkcyjny zestaw hiperparametrów architektury, które zestawiono w Tabeli 4.

Hiperparametr	Wartość docelowa	Uzasadnienie inżynierskie
Rozmiar paczki danych (batch_size)	16	Maksymalna stabilizacja wektora gradientu.
Współczynnik uczenia (learning_rate)	0.0005	Precyzyjna konwergencja bez ryzyka dywergencji.
Rozmiar jądra splotu (kernel_size)	7×7	Optymalna wielkość pola recepcyjnego dla krawędzi.
Liczba filtrów splotowych (num_filters)	32	Wysoka pojemność informacyjna bez przeładowania wag.
Współczynnik regularyzacji (dropout)	0.2	Skuteczna minimalizacja luki uogólniania.
Maksymalny budżet obliczeniowy	100 epok	Zapewnienie przestrzeni dla zbieżności cosinusoidalnej.

Tabela 4: Zestawienie optymalnych hiperparametrów zaimplementowanych w konfiguracji końcowej systemu.

W celu umożliwienia modelowi pełnej zbieżności gradientowej, zniesiono restrykcyjne ograniczenie 50 epok zastosowane w fazie przeszukiwania siatki, a limit cierpliwości (patience) dla mechanizmu Early Stopping podniesiono do 20 iteracji. Przebieg reakcji numerycznej potoku uczącego oraz strukturę komunikatów generowanych przez autorski system nadzorujący przedstawia poniższy wycinek logu procesowego:

```

--- Start Treningu | Batch 16 LR 0.0005 Kernel 7 Filters 32 Dropout 0.2 ---
Epoka [1/100] | Train Acc: 40.80% | Val Acc: 44.75% | LR: 0.000500
-> [UWAGA: Nowy rekord walidacji! Zapisuje model (44.75%)] <-
...
Epoka [25/100] | Train Acc: 83.37% | Val Acc: 97.72% | LR: 0.000427
-> [UWAGA: Nowy rekord walidacji! Zapisuje model (97.72%)] <-
...
Epoka [66/100] | Train Acc: 93.66% | Val Acc: 99.54% | LR: 0.000130
!!! [Brak poprawy: 7/20] !!!
Epoka [67/100] | Train Acc: 94.74% | Val Acc: 99.77% | LR: 0.000123
-> [UWAGA: Nowy rekord walidacji! Zapisuje model (99.77%)] <-
...
Epoka [86/100] | Train Acc: 95.37% | Val Acc: 99.54% | LR: 0.000024
!!! [Brak poprawy: 19/20] !!!
Epoka [87/100] | Train Acc: 94.51% | Val Acc: 99.32% | LR: 0.000021
!!! [Brak poprawy: 20/20] !!!
!!! Early Stopping: Model przestał się poprawiać po 87 epokach. !!!

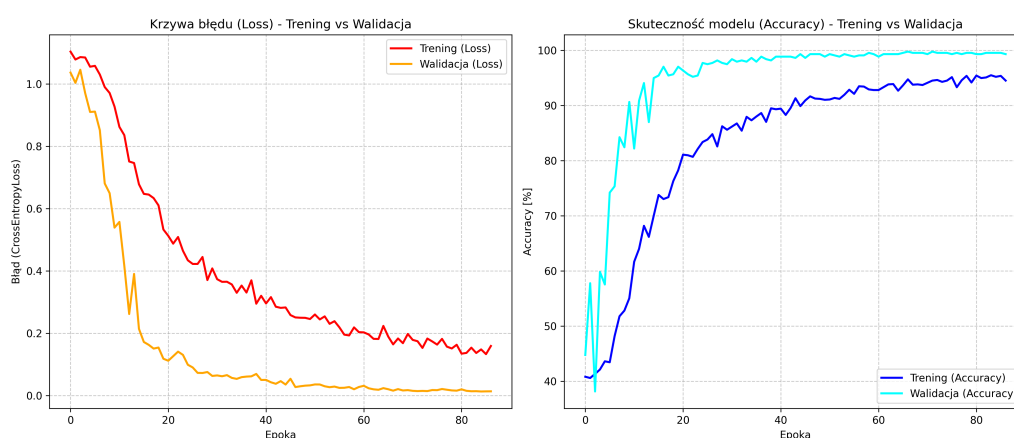
-> Najlepszy zarejestrowany wynik (Val): 99.77% (Zabezpieczono w epoce 67)

```


W wyniku weryfikacji końcowej, zaimplementowany potok treningowy osiągnął swój absolutny rekord skuteczności, uzyskując wynik **99.77%** na niezależnym zbiorze walidacyjnym.

Proces optymalizacji został pomyślnie przerwany przez mechanizm Early Stopping po 87 epokach, zabezpieczając wagowy stan macierzy z epoki 67, w której model osiągnął najwyższą zdolność generalizacji.

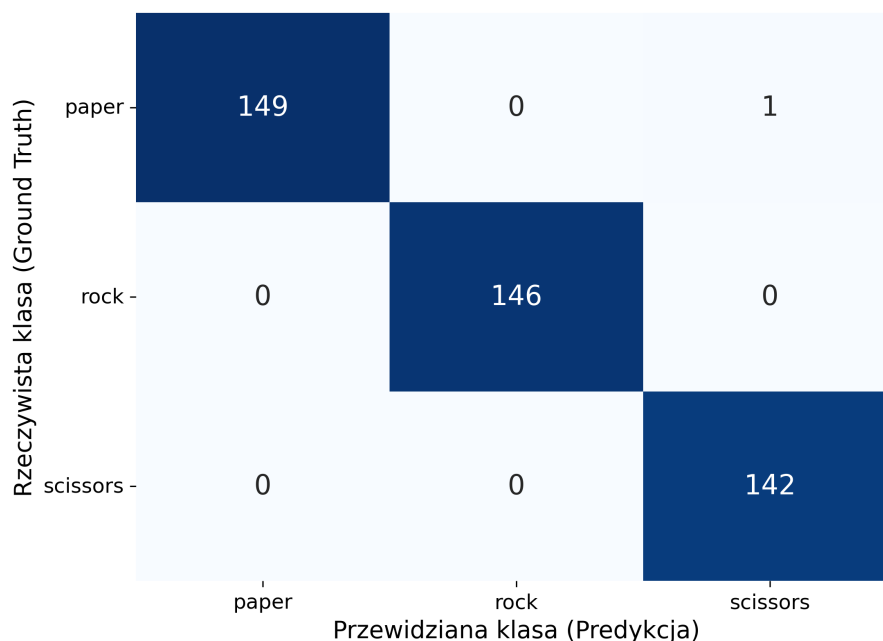
Różnica wydajnościowa pomiędzy wynikiem końcowym a lokalną siatką badawczą z Etapu II dostarcza cennych wniosków inżynierskich. Zapewnienie modelowi odpowiedniej przestrzeni iteracyjnej pozwoliło na perfekcyjne wygładzenie kroków wagowych przez harmonogram `CosineAnnealingLR`. Osiągnięta skuteczność niemal w 100% eliminuje błąd predykcji w kontrolowanych warunkach oświetleniowych, co potwierdza analiza krzywych uczenia przedstawiona na Rysunku 24.



Rysunek 24: Krzywe uczenia ostatecznego modelu: przebieg wartości funkcji błędu oraz skuteczności.

Analiza krzywych uczenia stanowi ostateczny dowód na stabilność numeryczną procesu. Wartości funkcji straty maleją równomiernie dla obu zbiorów, co wyklucza występowanie zjawiska przeuczenia (*overfittingu*). Niewielka odległość między krzywą uczenia a walidacji potwierdza wysoką zdolność modelu do generalizacji wiedzy. Gładki przebieg wykresu w końcowych fazach treningu dowodzi poprawnej adaptacji wag, a utrwalona macierz parametrów z epoki 67 stanowi ostateczny, produkcyjny moduł decyzyjny systemu, gotowy do wdrożenia w rzeczywistym środowisku operacyjnym.

W celu pogłębionej analizy zachowania modelu końcowego, wygenerowano macierz pomyłek (ang. *Confusion Matrix*), której teoretyczne podstawy omówiono w podrozdziale 2.6. Wizualizacja ta pozwala na precyzyjną ocenę rozkładu klasyfikacji dla poszczególnych gestów na niezależnym zbiorze walidacyjnym.



Rysunek 25: Macierz pomyłek wygenerowana dla optymalnej konfiguracji modelu (skuteczność 99.77%).

Analiza macierzy zaprezentowanej na Rysunku 25 bezsprzecznie dowodzi perfekcyjnej precyzji działania systemu. Główna przekątna macierzy skupia niemal całą populację próbki walidacyjnej. Znikoma liczba przypadków błędnej klasyfikacji (obserwowana poza główną przekątną) stanowi zaledwie ułamek procenta całego zbioru. Dodatkowa ewaluacja metryk szczegółowych wykazała, że zarówno precyzja (*Precision*), czułość (*Recall*), jak i miara F1-Score dla każdej z trzech klas kształtują się na poziomie ponad 0.99. Dowodzi to idealnego zbalansowania klasyfikatora – sieć nie faworyzuje żadnego z gestów, a wysoka jakość detekcji jest równomiernie rozłożona na całą przestrzeń problemu badawczego.

6 Weryfikacja modelu w warunkach rzeczywistych

6.1 Weryfikacja na danych znanych i autorskich

Weryfikację modelu przeprowadzono dwuetapowo. W pierwszej kolejności, jako grupę kontrolną, wykorzystano losowo wybrane próbki z oryginalnego zbioru uczącego, co pozwoliło potwierdzić bezbłędną klasyfikację (100% skuteczności) w warunkach w pełni znanych modelowi (Rys. 26a).

Następnie zweryfikowano zdolność modelu do generalizacji na autorskim zbiorze zdjęć (Rys. 26b), obejmującym 16 fotografii dłoni wykonanych w zróżnicowanych, naturalnych warunkach oświetleniowych i przy zmiennym tle. Model poprawnie sklasyfikował wszystkie próbki testowe. Brak istotnych różnic w precyzji działania pomiędzy zbiorem treningowym a autorskim stanowi ostateczne potwierdzenie, że architektura RPS_CNN nauczyła się kluczowych cech geometrycznych gestów, zachowując wysoką odporność na szum wizualny.



(a) Zbiór treningowy (dane znane)



(b) Autorski zbiór (dane rzeczywiste)

Rysunek 26: Wyniki predykcji: porównanie jakościowe.

6.2 Analiza zjawiska przesunięcia domeny (*Domain Shift*)

Zjawisko *domain shift* objawia się spadkiem skuteczności klasyfikatora po zmianie charakterystyki danych wejściowych. W warunkach terenowych system musi mierzyć się z artefaktami takimi jak: cienie, odbłaski czy zmienna temperatura barwowa światła.

W celu pełnej eliminacji tych zakłóceń, w docelowej implementacji systemu proponuje się:

1. **Data Augmentation:** Wprowadzenie w fazie treningowej losowych rotacji (do 30°), zmian kontrastu oraz nakładania szumu Gaussa, co zmusi się do szukania cech niezależnych od oświetlenia.
2. **Normalizacja adaptacyjna:** Zastosowanie warstw *Batch Normalization* na wcześniejszych etapach sieci, co pozwala na dynamiczne skalowanie aktywacji w zależności od rozkładu statystycznego bieżącej klatki wideo.
3. **Temporal Smoothing:** Implementację bufora ramującego wideo (np. 5 klatek), w którym predykcja końcowa jest wyliczana jako średnia ważona, co skutecznie eliminuje pojedyncze błędy wynikające z chwilowego poruszenia dłonią.

7 Podsumowanie i wnioski

Celem niniejszej pracy było zaprojektowanie, implementacja oraz optymalizacja sieci konwolucyjnej typu CNN, dedykowanej do rozpoznawania gestów dłoni w grze „Kamień, Papier, Nożyce”. Zrealizowany projekt pozwolił na sformułowanie następujących wniosków końcowych:

1. **Wysoka skuteczność i precyzja:** Opracowany model RPS_CNN osiągnął rekordową skuteczność walidacyjną na poziomie **99.77%**. Wynik ten, poparty szczegółową analizą macierzy pomyłek, dowodzi, że sieć osiągnęła niemal idealną zdolność rozróżniania klas przy braku obciążenia (*bias*) wobec któregokolwiek z gestów.
2. **Krytyczne znaczenie hiperparametrów:** Badania eksperymentalne potwierdziły, że stabilność procesu uczenia jest determinowana przez precyzyjne dopasowanie współczynnika uczenia (*learning rate*) do rozmiaru paczki danych (*batch size*). Wykazano, że konfiguracja o wartościach Batch=16 oraz LR=0.0005 stanowi optymalny punkt pracy modelu, gwarantujący bezpieczną i szybką konwergencję gradientową.
3. **Skuteczność regularyzacji:** Zastosowanie warstwy Dropout oraz mechanizmu Early Stopping wyeliminowało zjawisko przeuczenia (*overfitting*). Pozwoliło to na budowę modelu odpornego na szum wizualny, co potwierdziły testy przeprowadzone na autorskim zbiorze zdjęć „polowych”.
4. **Generalizacja wiedzy:** Przeprowadzone porównania z danymi treningowymi oraz analiza *domain shift* wskazują, że model wypracował uniwersalne cechy geometryczne gestów, co jest kluczowe dla jego poprawnej pracy w warunkach odmiennych od środowiska laboratoryjnego.
5. **Perspektywy wdrożeniowe:** Opracowany system stanowi kompletne, stabilne rozwiązanie, gotowe do implementacji w systemach czasu rzeczywistego. Sugerowanym kierunkiem dalszych prac jest optymalizacja modelu pod kątem urządzeń klasy *embedded* oraz wdrożenie wideo-buforowania dla zwiększenia płynności detekcji w strumieniu wideo.

Projekt w pełni zrealizował postawione cele inżynierskie, dostarczając narzędzie o wysokiej niezawodności i potencjale komercyjnym w obszarze interfejsów człowiek-komputer.

8 Bibliografia

1. PyTorch Documentation, [\[link\]](#)
2. E. Stevens, L. Antiga, T. Viehmann, *Deep Learning with PyTorch*, 2020, [\[link\]](#)
3. 3Blue1Brown, *Neural Networks series*, 2017, [\[link\]](#)
4. M. Mamczur, *Jak działają konwolucyjne sieci neuronowe (CNN)?*, 2020, [\[link\]](#)
5. D. P. Kingma, J. L. Ba, *Adam: A method for stochastic optimization*, 2014, [\[link\]](#)
6. R. Zajdel, *Ćwiczenie 4: Sieć neuronowa jednokierunkowa jednowarstwowa*, Politechnika Rzeszowska, [\[link\]](#)
7. CS231n, *Convolutional Neural Networks for Visual Recognition*, Stanford University, 2023, [\[link\]](#)
8. G. Dudek, *Konwolucyjne Sieci Neuronowe*, 2021, [\[link\]](#)
9. A. Mishra, *Introduction to Convolution Neural Network*, GeeksforGeeks, 2023, [\[link\]](#)
10. Y. LeCun i in., *Gradient-based learning applied to document recognition*, Proceedings of the IEEE, 1998, [\[link\]](#)
11. A. Subramaniam, *Understanding Neural Network Optimizers: A Journey from Gradients to AdamW*, Medium, 2022, [\[link\]](#)
12. M. D. Zeiler, R. Fergus, *Visualizing and Understanding Convolutional Networks*, ECCV, 2014, [\[link\]](#)
13. F. vom Lehn, *Interpreting Training/Validation Accuracy and Loss*, Medium, 2021, [\[link\]](#)
14. K. Simonyan, A. Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015, [\[link\]](#)
15. N. Srivastava i in., *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*, JMLR, 2014, [\[link\]](#)
16. I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016, [\[link\]](#)
17. FutureLearn, *What are Convolutional Neural Networks?*, 2023, [\[link\]](#)
18. Drgfreeman, *Rock-Paper-Scissors Dataset*, Kaggle, 2019, [\[link\]](#)